



TM270

Configuration, commissioning and diagnosis of mapp services

Requirements

Completed trainings	SEM210 Automation Studio training: Basics SEM611 Automation Studio training: Creating an HMI application with mapp View
Software	Automation Studio 4.9.1 Automation Runtime 4.91 mapp Services 5.14 mapp View 5.14
Licenses	1TCMPSERVICE.10-01 mapp Services premium
Hardware	ARsim or X20 CPU

Table of contents

1 Introduction.....	4
1.1 Learning objectives.....	4
1.2 Symbols and safety notices.....	4
1.3 B&R online courses.....	5
2 mapp Technology.....	6
3 mapp Services.....	8
4 Recipe management with mapp Recipe.....	10
5 Data logging with mapp Data.....	13
6 System of units.....	15
7 Alarm management with mapp AlarmX - Part I.....	17
7.1 Alarm system terminology.....	17
7.2 Alarm behavior.....	18
7.3 Configuring the alarm system.....	19
7.4 Text system.....	22
8 Diagnostics via WebXs.....	27
9 Hierarchy with mapp Com - Part I.....	28
10 Connecting to an HMI application: User interface vs. mapp View.....	29
11 Event management with mapp Audit.....	30
12 User management with mapp UserX.....	35
13 File management system with mapp File.....	39
14 Creating reports with mapp Report.....	41
15 Alarm management with mapp AlarmX - Part II.....	43
15.1 Severity.....	43
15.2 Alarm behavior \ Alarm monitoring.....	44
15.3 Mapping alarms.....	45
15.4 Displaying alarms in the HMI application.....	46
16 Hierarchy with mapp Com - Part II.....	50
17 Summary.....	52

1 Introduction

mapp is revolutionizing the creation of software for industrial machinery and equipment. mapp components are as easy to use as smartphone apps.

Rather than write lines and lines of code to build a user management system, alarm system or motion control sequence from the ground up, developers of machine software simply configure the ready-made mapps with a few clicks of the mouse. Complex algorithms are easy to master.

Programmers can focus entirely on the machine process.



Figure 1: mapp Technology - The future of software development

1.1 Learning objectives

This training module explores the mapp Services Technology Package in more detail. It provides general information about how and when to use some of the mapp components for this area of technology. There are numerous exercises available to help increase understanding. Automation Help and the B&R Tutorial Portal serve as the basis for implementing the exercises.

- Participants will understand the concept of mapp Technology and the benefits of its use.
- Participants will become familiar with the different mapp Technology Packages and their application areas.
- Participants will become familiar with the range of functions of mapp Services and will be able to locate system requirements, version information, licensing and use cases.
- Participants will become familiar with Automation Help and B&R tutorials and how to use them.
- Participants will be able to configure and call mapp Services components.
- Participants will be able to display, operate and diagnose mapp Services components.
- Participants will be able to load and save recipe data.
- Participants will be able to log machine data and will become familiar with the different recording options.
- Participants will develop an understanding of the relationships between mapp components, how they can be combined and how they are managed in the system.
- Participants will be able to access and load files with the file explorer.
- Participants will be able to manage their own alarms in the alarm system and display and filter alarms that have been triggered.
- Participants will learn how to use the user role system and the system of units in the project.
- Participants will be able to record and display user events that have occurred.
- Participants will learn how to use the text system in the project and where and how to apply it.
- Participants will be able to perform an extensive diagnosis on the mapp components.
- Participants will be able to create individual reports.
- Participants will be able to build a hierarchy of the alarms from different machine groups.

1.2 Symbols and safety notices

Unless otherwise specified, the symbol descriptions and safety notices listed in "TM210 - Working with Automation Studio" apply.

1.3 B&R online courses

The B&R online courses provide lessons for a wide range of topics. Because the courses are interactive, they allow content to be learned effectively.

To help you find the B&R online courses you need more quickly, the various courses on the website are assigned to different training categories and are based on the modular training concept.

[B&R online courses](https://www.br-automation.com/en/academy/online-courses/) <https://www.br-automation.com/en/academy/online-courses/>)



2 mapp Technology

mapp Technology stands for modular application technology and is an umbrella brand for B&R products.

It allows extensive functionalities to become easily accessible. Many complex operations, such as loading and saving recipe data, controlling a drive axis and recording process values, can be implemented quickly and easily using mapp Technology components.

mapp Technology unites configuration and programming. The functionality itself is implemented in the application program using standard libraries. In addition, mapp provides configuration interfaces. Similar to hardware modules, these are used to configure the functionality of the mapp components without programming.

mapp Technology is divided into different areas.

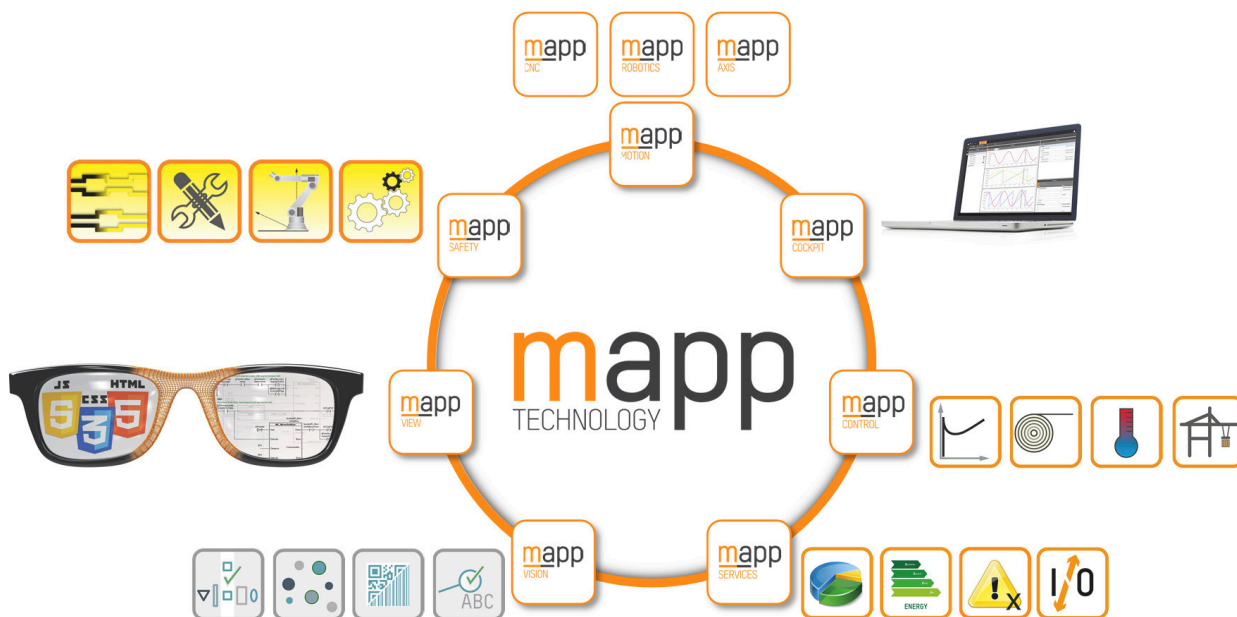


Figure 2: mapp Technology brand

Package	Description
mapp SERVICES	Set up all basic functions for a machine or system with just a few clicks: recipe system, alarm system, OEE evaluation, user-role system, audit trail system, energy monitoring, database system and much more.
mapp MOTION	mapp Motion provides uniform solutions for all areas of motion control: from individual axes to multi-axis systems and even complex robotics and CNC applications.
mapp VIEW	The only HMI solution on the market that works independently of platform and operating system. Modern HTML5 applications can be created with ready-made widgets.
mapp CONTROL	Complex control algorithms in the form of easy-to-use software blocks. Crane control, hydraulics control, filter design, closed-loop design and much, much more. Advanced technology made accessible for the average user.

Table 1: Overview of mapp Technology Packages

Package	Description
	Maximum productivity through integrated safety technology. mapp Safety covers the entire spectrum, including safe axes and robots. Safe machine options can be enabled or disabled in the field.
	Web server for diagnosing and commissioning various components, e.g. all mapp Motion objects.
	Includes preconfigured functions, advanced image processing algorithms and easy configuration.

Table 1: Overview of mapp Technology Packages

User access to mapp Technology is enabled via independent Technology Packages. These are loaded from the B&R website or via the Automation Studio upgrades dialog box and provide various elements for Automation Studio.

Elements of a mapp Technology Package:

- Libraries
- Configuration element in Configuration View
- Toolbox elements
- Corresponding Automation Help entries
- Automation Studio editors

New versions of all technology packages are released every quarter. The versions are structured according to the following scheme.



Figure 3: Version information for mapp Technology packages

**Which versions fit together?**

Only versions that have the same major and minor version are compatible. A Technology Package can only be upgraded if all other packages are upgraded to the same basic versions.

3 mapp Services

As a subelement of the mapp technology, the components of mapp Services provide easy access to the extensive basic functionalities of a machine. Many complex operations, such as loading and saving recipe data, extensive alarm handling or recording process values, can be implemented quickly and easily using mapp Services components.

Below is an overview of some of the components of the mapp Services Technology Package:

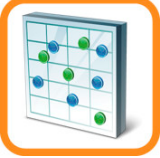
	Name	Description
	MpRecipeXml	mapp Recipe provides all of the functions necessary for simple yet high-speed recipe management. This component is compatible with all other mapp components and therefore acts as a centralized recipe management system that consolidates the parameters from across the entire machine infrastructure.
	MpData	mapp Data makes it possible for users to back up values of defined process variables (PVs). This data is stored in CSV files.
	MpAlarmX	mapp AlarmX collects and manages both mapp alarms and user alarms. The alarms are configured using Automation Studio, managed in the application and then displayed in an HMI application or exported as a file.
	MpUserX	mapp UserX sets up user management. Roles and users are created using the user role system in Automation Studio (OPC UA compliance included) and then managed using MpUserX. This includes access rights, user data, password, logging in/out and connecting to the HMI application.
	MpAudit	mapp Audit can log different events. These can originate from the HMI application or user-defined events. The user can determine the format in which the events are stored in a file.
	MpFile	mapp File provides a file management system as well as a connection to the HMI application to display files.
	MpReport	mapp Report creates PDF reports that can be exported to a storage medium.

Table 2: Overview of some of the mapp components

	Name	Description
	MpIO	Hardware configurations and the IO mapping can be changed at runtime using the mpIO component.
	MpCodeBox	mapp CodeBox can be used to create and execute programs on the machine at runtime. Programs are created using a component in the mapp View HMI application that allows the programming of ladder diagrams.

Table 2: Overview of some of the mapp components

4 Recipe management with mapp Recipe

The MpRecipe component provides all functions necessary for quick and simple recipe management as well as reading and writing recipe files. mapp Recipe is compatible with all other mapp components and therefore acts as a centralized recipe management system that consolidates the parameters from across the entire machine infrastructure.

The function block **MpRecipeXml** makes it possible to load parameters from a certain file or write parameters to that file.

The function block **MpRecipeRegPar** can be used to register a PV as a recipe parameter.

The function block **MpRecipeUI** provides a complete interface for visualization in order to display a list of possible recipes. The MpFile library must be transferred to the controller in order to use this function block.



[Services \ mapp Services \ mapp Recipe: Recipe management \ Concept](#)
[Services \ mapp Services \ mapp Recipe: Recipe management \ Libraries \ Function blocks \ MpRecipeUI](#)

Creating a recipe management system with MpRecipeXml

Exercise: Configure a file device as storage location for a recipe

To store data as files in one place, a file device is created.

- 1) CPU configuration \ Click "File devices".
- 2) Enter any name in the "Name" field, e.g. CF, Location.
The length of the name is limited to 127 characters.
- 3) In the "Path" field, enter the path to the file system.
 - If the simulation is used: e.g. C:\Temp\.
 - If a real X20 is used: e.g. F:\. In addition, "Module system on target" \ "Minimum user partition size" = 10 must be set to a suitable value and "Module system on target" = "SAFE" must be defined.



The drive must be specified in capital letters (however, this does not apply to the spelling of the directory) and finished with a backslash "\".



[Programming \ Editors \ Configuration editors \ Hardware configuration \ CPU configuration \ SG4 \ CPU properties - File devices](#)

Exercise: Prepare recipe management with MpRecipeXml

The goal of this exercise is to create a functional recipe management system with function blocks "MpRecipeXml" and "MpRecipeRegPar".

First, the mapp Recipe component is added and edited. Then the structure is created, which serves as the recipe.

- 1) Under "mapp Services" \ "IceCreamMachine", add the file "MpRecipeXml Default" and name it "IceRecipe".
- 2) Rename "MpLink" as "MpLinkICEgRecipeXml".
- 3) Add "ST Program All In One" and name it "Ice_Recipe".
- 4) Create the following structure in "Types.typ":

TYPE

```
IceCreamIngredients_Type :      STRUCT  (*Structure of the ice cream recipe*)
    Milk : REAL := 200.0; (*Amount of milk in [ml]*)
    Cream : REAL := 250.0; (*Amount of cream in [ml]*)
    Sugar : REAL := 60.0; (*Amount of sugar in [g]*)
    EggYolk : USINT := 3; (*Amount of egg yolk in pieces*)
    Flavour : STRING[80] := 'Chocolate'; (*Flavour of the ice cream*)
```

```
END_STRUCT;
END_TYPE
```

5) Create the following variable in "Variables.var":

```
VAR
IceFormula : IceCreamIngredients_Type; (*Variable with the structure
IceCreamIngredients_Type*)
END_VAR
```

The function blocks are then added and configured. Test the program in the Watch window.

1) Use function block "MpRecipeXml_0" in the initialization subroutine:

- Enable - True
- MpLink - ADR(MpLinkICEgRecipeXml)
- DeviceName - ADR('CF')
- FileName - ADR('Icecream_recipe')

2) Use function block "MpRecipeRegPar_0" in the initialization subroutine:

- Enable - True
- MpLink - ADR(MpLinkICEgRecipeXml)
- PVName - ADR('Ice_Recipe:IceFormula')

3) Call the function blocks in the cyclic program section:

- MpRecipeXml_0()
- MpRecipeRegPar_0()

4) Use and call the function block "MpRecipeXml_0" in the exit program:

- Enable - False
- MpRecipeXml_0()

5) Use and call the function block "MpRecipeRegPar_0" in the exit program:

- Enable - False
- MpRecipeRegPar_0()

6) Enable monitor mode and test the program. Display MpRecipeXml_0 and IceFormula in the Watch window.

- MpRecipeXml_0 - Save - True
- Open the file created in the target storage location.
- Change and save the values in the structure.
- MpRecipeXml_0 - Save - False
- MpRecipeXml_0 - Load - True



The recipe is placed in the configured storage location and can be opened. If these values are overwritten and saved, the recipe can be loaded into the project with "Load". This overwrites the parameters in structure "IceFormula".

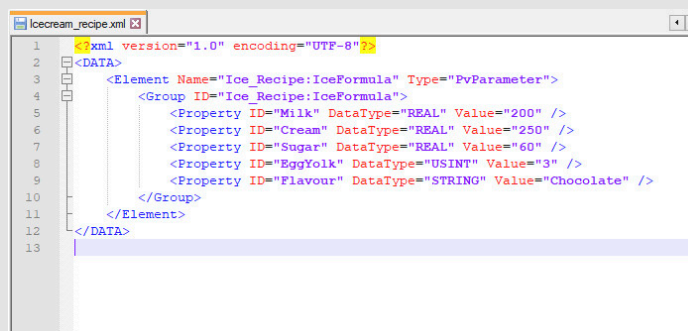


Figure 4: Saved ice cream recipe, stored in the configured storage location.

Communicating with recipe management via the user interface

Exercise: Communicate with recipe management via the user interface

The goal of this exercise is to communicate with recipe management via the user interface. Operate the user interface via the Watch window.

First, the UI function block and associated interfaces are added along with the library.

- 1) Create the following variables in "Variables.var":

```
(*UI-Connection*)
VAR
MpRecipeUI_0 : MpRecipeUI;
(*MpRecipeUI for UI Connection to the ice cream machine*)
UIConnect : MpRecipeUIConnectType;
(*UI Connection - contains the parameters needed for the connection*)
END_VAR
```

- 2) Add library "MpFile".

The function blocks are then added and configured. Test the program in the Watch window.

- 1) Use function block "MpRecipeUI_0" in the initialization subroutine:

- Enable - True
- MpLink - ADR(MpLinkICEgRecipeXml)
- UIConnect - ADR(UIConnect)

- 2) Call the function block in the cyclic program section:

- MpRecipeUI_0()

- 3) Use and call the function block "MpRecipeUI_0" in the exit program:

- Enable - False
- MpRecipeUI_0()

Now communication is possible via the user interface.

- 1) In UIConnect, search for 'Icecream_recipe.xml' under "Recipe \ List \ Names".
- 2) Remember the corresponding index.
- 3) Enter the index in "SelectedIndex".
- 4) Set Load to True.

5 Data logging with mapp Data

mapp Data makes it possible for users to back up values of defined process variables (PVs). This data is stored in CSV files.

The function block **MpDataRecorder** logs PVs and stores them in a CSV file on the file device. There is an enumeration on the RecordMode input that describes in which mode new information should be stored in a file, e.g. time-based, after a trigger, with a value change.

The function block **MpDataRegPar** registers PVs for data logging.



[Services \ mapp Services \ mapp Data: Data logging \ Concept](#)

Creating a data recording using MpDataRecorder

Exercise: Create a data recording using MpDataRecorder

The goal of this exercise is to monitor the temperature of the ice cream and record the data. Function blocks "MpDataRecorder" and "MpDataRegPar" are used. The recording mode is set so that each time the value of the variable changes, the new PV data is stored.

First, the mapp Data component is added and edited. Next, the program is created.

- 1) Under "mapp Services" \ "IceCreamMachine" add the file "MpDataRecorder" and name it "IceData".
- 2) Rename MpLink to "MpLinkICEgDataRecorder".
- 3) Add an "ST Program All In One" in the "IceCreamMachine" folder and name it "Ice_Data".
- 4) Create the following variable in "Global.var":

```
(*Ice cream machine*)
VAR
gIceCreamTemperature : REAL := -8;
(*Initial value of the temperature = -8 °C*)
END_VAR
```

The function blocks are then added and configured. The program is tested in the Watch window.

- 1) Use function block "MpDataRecorder_0" in the initialization subroutine:
 - Enable - True
 - MpLink - ADR(MpLinkICEgDataRecorder)
 - DeviceName - ADR('CF')
 - RecordMode - mpDATA_RECORD_MODE_VALUE
- 2) Use function block "MpDataRegPar_0" in the initialization subroutine:
 - Enable - True
 - MpLink - ADR(MpLinkICEgDataRecorder)
 - PVName - ADR('gIceCreamTemperatures')
- 3) Call the variable "gIceCreamTemperature" in the initialization subroutine.
- 4) Call the function blocks in the cyclic program section:
 - MpDataRecorder_0()
 - MpDataRegPar_0()
- 5) Use and call the function block "MpDataRecorder_0" in the exit program:
 - Enable - False
 - MpDataRecorder_0()

6) Use and call the function block "MpDataRegPar_0" in the exit program:

- Enable - False
- MpDataRegPar_0()

7) Enable monitor mode and test the program. Display MpDataRecorder_0 and glceCreamTemperature in the Watch window.

- MpDataRecorder_0 - Record - True
- Change glceCreamTemperature
- Open the file created in the target storage location.



The data recording is placed in the configured storage location and can be opened. Any change to the value is recorded. The recording is reset with "Record - True". None of the previously generated data is taken into account!



If the recorded values are displayed incorrectly in Excel (e.g. in date format), a change must be made in the mapp Data configuration. The decimal separator can be adjusted here under "Decimal mark".

6 System of units

Automation Studio comes with a unit system. It contains a large selection of units. By default, a process variable does not have a unit. To display the process data including the unit, however, a unit must be defined for the process variable. This is done in the OPC UA configuration.

The variable that should be assigned a unit is enabled in the OPC UA default view editor. Next, a unit is assigned to the variable via the Toolbox. The OPC UA server must be enabled in the configuration of the target system for this.

The unit assigned via OPC UA is called "**engineering unit**" or "E-unit" and describes the unit in which the process variable is processed in the **application**.

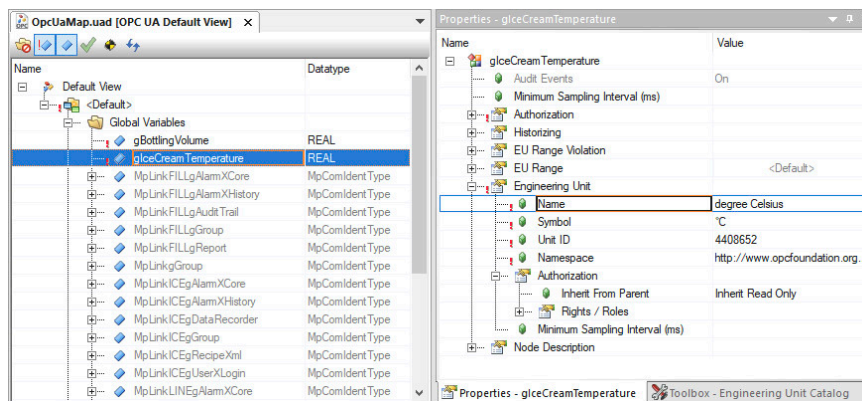


Figure 5: The variable "gIceCreamTemperature" is enabled and has the unit "degree Celsius".

There is also the "**display unit**" or "D-unit". This setting expands the unit for **HMI applications** and **file exports**. The display unit must always be defined for all possible unit systems (metric, imperial and imperial-us).

All units including their CommonCode (e.g. degrees Celsius = CEL) are defined in Automation Help.

The display unit can either be defined directly in the widget used in the HMI application, or the MpComUnit configuration can be used. The latter is clearer and easier to implement for larger applications with many units.

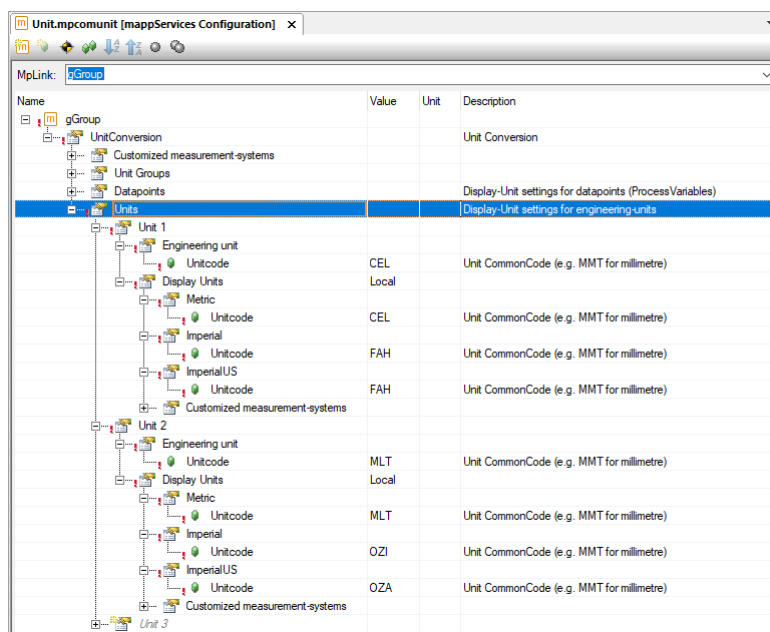


Figure 6: All variables with the engineering units CEL and MLT are additionally assigned the units for metric, imperial and imperial-us as the display unit.

If the display unit is not defined for a process value, then the engineering unit is used. If this is also missing, then the variable will be displayed without a unit.



[Services \ mapp Services \ mapp Com: mapp management \ Concept \ Unit management](#)
[Services \ mapp Services \ mapp Data: Data logging \ Use cases \ Recording data with a unit](#)
[Communication \ OPC UA \ Configuration in AS \ OPC UA default view configuration \ OPC UA default view editor \ Engineering Unit Catalog](#)
[Programming \ Unit system](#)

Creating a data recording using the corresponding unit

Exercise: Create a data recording using the corresponding unit

The goal of this exercise is to record the temperature in the room where the ice cream machine is located using the appropriate unit.

First, the engineering unit is defined.

- 1) Under "Connectivity" \ "OpcUA", add the file "OPC UA Default View File".
- 2) Under "Global variables", right click on "glceCreamTemperature" and click on "Enable tag".
- 3) Search for "degree Celsius" in the toolbox.
- 4) Drag and drop the "degree Celsius" entry with symbol "°C" onto the "glceCreamTemperature" tag.

The MpComUnit component is then added and the display unit is defined in it.

- 1) Under "mapp Services", add the file "MpComUnit" and name it "Unit".
- 2) Under "Unit conversion" \ "Units" \ "Unit 1" \ "Engineering unit", enter "CEL" in the "Unit code" field.
 - ° Under "Unit 1" \ "Display units" \ "Metric", enter "CEL" in the "Unit code" field.
 - ° Under "Unit 1" \ "Display units" \ Imperial", enter "FAH" in the "Unit code" field.
 - ° Under "Unit 1" \ "Display Units" \ ImperialUS", enter "FAH" in the "Unit code" field.
- 3) Under "mapp Services" \ "IceCreamMachine", open and edit the file "IceData":
 - ° Under "DataRecorder" \ "Unit definition", select the option "Measurement-system based".
 - ° Under "DataRecorder" \ "Unit definition" \ "Unit display", select the option "Full name".
 - ° Under "DataRecorder" \ "Unit definition" \ "Measurement system", select the option "Metric".
- 4) Re-export data recording from task "Ice_Data" and analyze the results.



If the recorded values are displayed incorrectly in Excel (e.g. in date format), a change must be made in the mapp Data configuration. The decimal separator can be adjusted here under "Decimal mark".

7 Alarm management with mapp AlarmX - Part I

Alarms are used to monitor and display specific machine states so that the users of those machines can respond appropriately and make the necessary decisions. The MpAlarmX component provides all of the functions necessary for simple yet extensive alarm handling.

MpAlarmX collects and manages both mapp alarms and user alarms. The alarms are configured with Automation Studio, managed in the application and then displayed in an HMI application or exported as a file.

The alarm system runs on the controller on a standalone basis. The mapp View "AlarmList" widget facilitates the process of creating an HMI application for alarms.

Other mapp components can be configured so that their alarms are automatically transferred to the MpAlarmX component. Communication between the individual components takes place via the MpLink, which is implemented by adding the standard configuration for mapp AlarmX in the Configuration View.

MpAlarmX even offers the possibility to implement modular alarm handling. This means that several hierarchically structured alarm systems can be created, and that for each alarm, the influence of the alarm on the higher-level alarm list can be defined.



[Services \ mapp Services \ mapp AlarmX: Alarm management \ Concept](#)
[Services \ mapp Services \ mapp AlarmX: Alarm management \ Configuration \ MpAlarmXCore configuration](#)
[Services \ mapp Services \ mapp AlarmX: Alarm management \ Libraries \ Function blocks \ MpAlarmX-Core](#)
[Services \ mapp Services \ mapp AlarmX: Alarm management \ Libraries \ Function blocks \ MpAlarmXListUI](#)

Features of the alarm system:

- Collecting mapp alarms and user alarms
- Recording active and historic alarm states
- Managing alarm texts and properties via the mapp configuration
- Binding the alarm system and text system via the configuration
- Consolidating different alarms based on "reactions"
- Exporting historic alarms to the file system
- Binding the alarm system to the "AlarmList" widget via mapp Link binding quickly and easily

7.1 Alarm system terminology

Errors, user-defined alarms and mapp alarms

- **Error**
Errors are indicated on the output of a function or function block via the "StatusID". When developing the application, the "StatusID" can be used for troubleshooting. Errors are displayed in the Logger.
- **User alarms**
The application developer can define specific alarms via the MpAlarmXCore configuration. The behavior of each alarm is also customized here. User alarms are triggered via the application.
- **mapp alarms**
mapp alarms can be found in the configuration for the respective components. In the configuration, the user must specify whether they want the alarms to be transferred to the alarm system.

Basic alarm state

An alarm can have the following states:

- **Active and not acknowledged**
The alarm is active. mapp alarms are enabled by the respective component, whereas user-defined alarms are set using MpAlarmXSet. If an alarm has not been acknowledged yet, it will have the state "Not acknowledged".
- **Active and acknowledged**
The alarm is active. An active alarm can also be already acknowledged. Alarms can be acknowledged in the application via MpAlarmXAcknowledge or in the HMI application.
- **Inactive and not acknowledged**
The alarm is not active and has not been acknowledged.
- **None**
The alarm has been disabled.

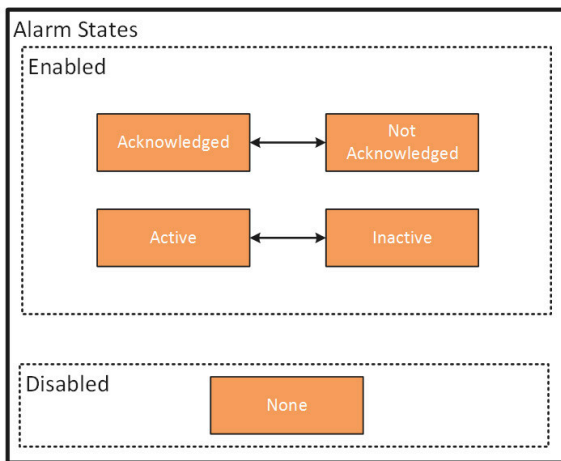


Figure 7: Possible alarm states

7.2 Alarm behavior

The behavior of an alarm is defined in the alarm configuration. There are two predefined types of behavior. One of these is "edge alarms", which represent short-term alarms, i.e. an event. "Persistent alarms" are also possible, which remain active until a certain condition is met, i.e. a state.

In addition, there are alarms for which set and reset conditions can be configured directly in the configuration. For a more detailed explanation of these monitoring alarms, see mapp AlarmX, Part II.

Edge alarms

An edge alarm is an alarm that is triggered for a short period only. It is set via the application.

Examples:

- "Unable to load recipe".
- "Unable to send SMS text message to Shift supervisor".
- "Storage medium not found".
- "Operation not permitted".

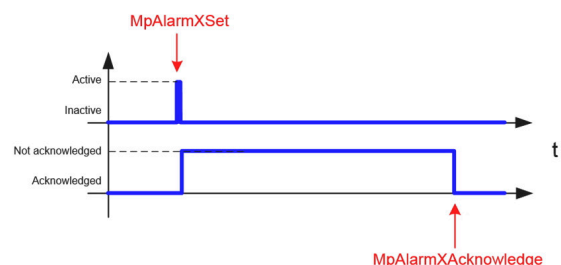


Figure 8: Edge alarm behavior

Persistent alarms

A persistent alarm is an alarm that is typically triggered for a longer period of time. It is set and reset via the application.

Examples:

- "The temperature (123°C) is not in the normal range (100 - 120°C)".
- "Tank water level too high".
- "Emergency switch-off pressed".
- "X20DI8371 not connected".

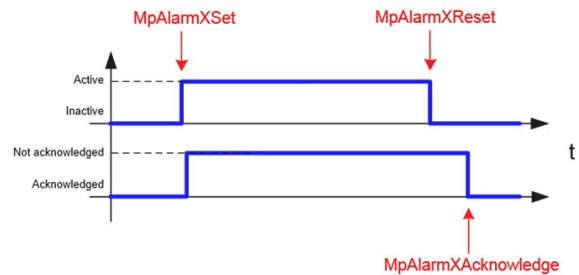


Figure 9: Persistent alarm behavior

7.3 Configuring the alarm system

The complete MpAlarmX alarm system consists of a mapp Services configuration and a corresponding function block.

In the first step, a new "MpAlarmXCore" mapp configuration is added to the Configuration View from the Toolbox in the "mapp Services" package.

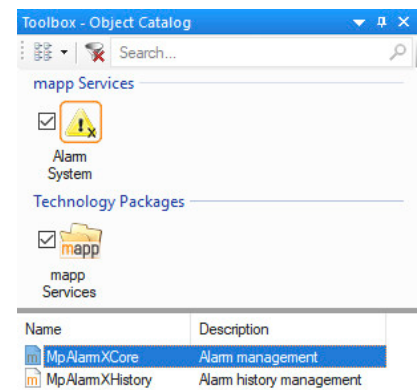


Figure 10: Selecting the "MpAlarmXCore" configuration from the Toolbox

Configuring a user alarm

The next step is to open the configuration and configure the first alarm. In the figure here, the "TemperatureHigh" alarm is configured as a "Persistent Alarm".

At runtime, user alarms are set and reset using the "MpAlarmXSet" and "MpAlarmXReset" functions in the MpAlarmX library.

Configurable alarm properties:

- Alarm number
- Severity
- Alarm text
- Alarm behavior
- Additional information

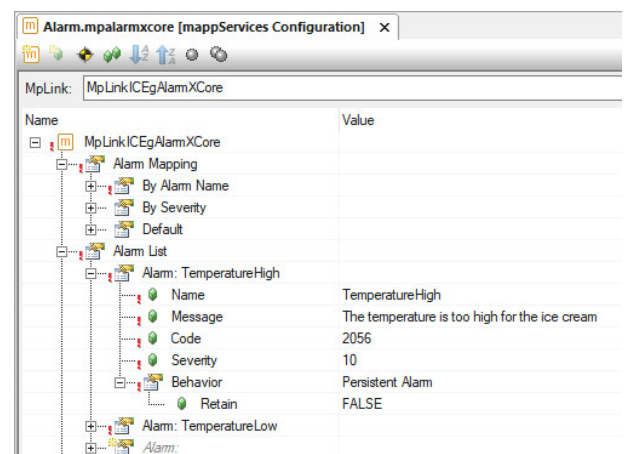


Figure 11: Configuration for "TempHigh" alarm

Starting the alarm system

The "MpAlarmXCore" function block is used to enable the alarm system on the controller. This function block manages all mapp alarms and user alarms at runtime. The previously created configuration is assigned to the function block via mapp Link.

Setting and resetting user alarms

User alarms are set and reset using the "MpAlarmXSet" and "MpAlarmXReset" functions respectively. User alarms are acknowledged using the "MpAlarmXAcknowledge" function. When the functions are called, the mapp Link is also transferred.

Handling of mapp alarms

The mapp concept is responsible for handling communication between mapp components. In some cases, other mapp components may have their own standalone alarms and need to transfer these to the alarm system. If applicable, this is enabled in the corresponding configuration. At runtime, mapp alarms are transferred to the alarm system automatically.

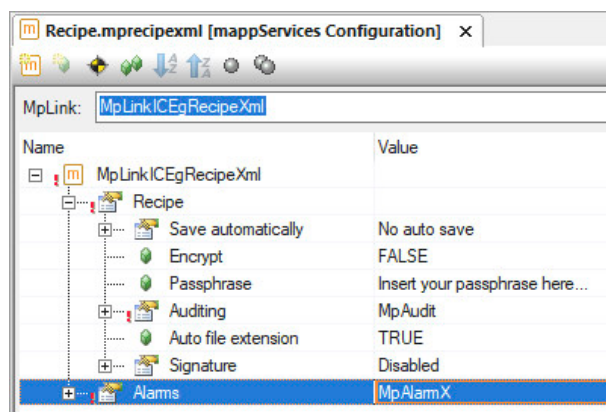


Figure 12: Alarms are sent to the MpAlarmX alarm system automatically.

Creating an alarm management system with MpAlarmX

Exercise: Create an alarm management system with MpAlarmX

The goal of this exercise is to create an alarm management system for the ice cream machine. Function block "MpAlarmXCore" is used at the start. Alarms are then triggered and reset by the user. First, the mapp AlarmX component is added and edited. Next, the program is created.

- 1) Under "mapp Services" \ "IceCreamMachine", add the file "MpAlarmXCore" and name it "IceAla".
- 2) Rename Mplink to "MplinkICEgAlarmXCore".
- 3) Enter "TemperatureHigh" in the "Name" field under "AlarmList".
 - Message - The temperature is too high for the ice cream
 - Code - 2056
 - Behavior - Persistent Alarm
- 4) Enter "TemperatureLow" in the "Name" field under "AlarmList".
 - Message - The temperature is too low for the ice cream
 - Code - 2066
 - Behavior - Persistent Alarm
- 5) Add an "ST Program All In One" in the "IceCreamMachine" folder and name it "Ice_Alarm".

The function block is then added and configured. The program is tested in the Watch window.

- 1) Use the function block "MpAlarmXCore_0" in the initialization subroutine:
 - Enable - True
 - Mplink - ADR(MplinkICEgAlarmXCore)
- 2) Call the function block in the cyclic program section and program the logic:

```
//ice cream temperature >= -4°C
IF gIceCreamTemperature >= -4 THEN
    MpAlarmXSet (MplinkICEgAlarmXCore, 'TemperatureHigh');
ELSE
    MpAlarmXReset (MplinkICEgAlarmXCore, 'TemperatureHigh');
END_IF;

//ice cream temperature <= -20°C
IF gIceCreamTemperature <= -20.0 THEN
    MpAlarmXSet (MplinkICEgAlarmXCore, 'TemperatureLow');
ELSE
```

```
MpAlarmXReset (MpLinkICEgAlarmXCore, 'TemperatureLow');  
END_IF;
```

3) Use and call the function block "MpAlarmXCore_0" in the exit program:

- Enable - False
- MpAlarmXCore_0()

4) Enable monitor mode and test the program. Display MpAlarmXCore_0 and glceCreamTemperature in the Watch window.

- Change glceCreamTemperature
- Monitor the status of ActiveAlarms and PendingAlarms



If the temperature of the ice cream is above -4°C or below -20°C, ActiveAlarms and PendingAlarms go to 1. If the temperature reaches a value between the two values, ActiveAlarms goes to 0 and PendingAlarms keeps its value. To reset PendingAlarms to 0, the alarm must be acknowledged using MpAlarmXAcknowledge.

7.4 Text system

The Automation Studio text system allows you to manage localizable texts for the target system centrally in an Automation Studio project. The integrated text system is used for localization of texts and the unit system is used to adjust the units.

Previously, alarm text was specified directly in the alarm configuration. However, the disadvantage here is that text entered in this way is excluded when switching languages. In addition to the direct use of alarm texts in the application and the display of these texts in the HMI application and in localized Logger entries, it is also possible for alarm texts to be localized via the text system.



[Services \ mapp Services \ mapp AlarmX: Alarm management \ Concept \ Alarm texts Programming \ Text system](#)

7.4.1 Localizing alarm text via the text system

Enabling the text system

First, the **project languages** are added to the Logical View. This is where the languages that are available for localization are managed.

The **text system configuration** is then added to the Configuration View. All project languages and text data that are required at runtime must be added to the text system configuration.

Finally, a **localizable text** is added to the Logical View. The alarm texts are added and edited here.

Namespaces and texts

Localizable texts are referenced globally in the project using the fully specified name, which consists of Text ID+Namespace. Namespaces define a logical hierarchy (similar to file paths) that is used to manage texts within a project independently of where the text modules are located.

The next step is to configure the namespace, text IDs and texts in the text file. If the texts are located in the "IAT" namespace, they will be localized later via the mapp View client. If they are located in a different namespace, they will be translated by the alarm system.

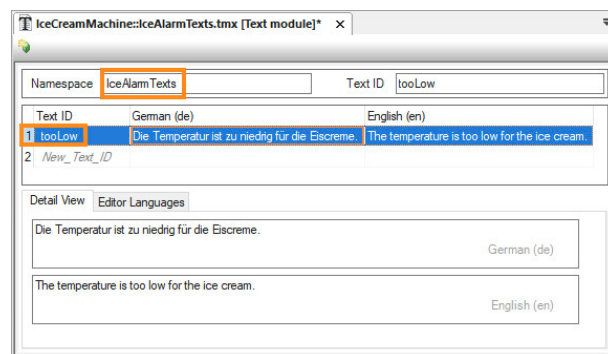


Figure 13: Entering the namespace in the text file

Using texts in the alarm configuration

In this figure, the alarm text is not directly specified in the "Message" parameter; instead, it is linked to the text system itself. The text source is a text file containing the localizable alarm texts that have been added to the Logical View.

Syntax: {\$Namespace/TextID}

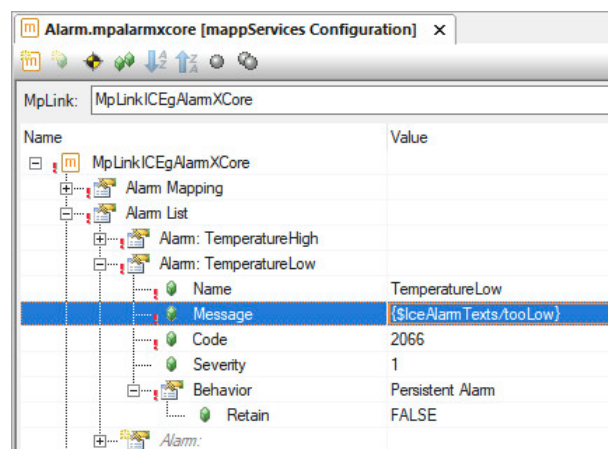


Figure 14: Referencing a text from the text system

7.4.2 Integrating application data

Alarm snippets are used to integrate application data directly into the alarm text. The definition can be found in the alarm configuration.

Alarm snippets can be used for different alarm texts. An alarm snippet can be used for one or more alarms. For each alarm snippet, a unique key is defined that identifies the alarm snippet in the alarm text.

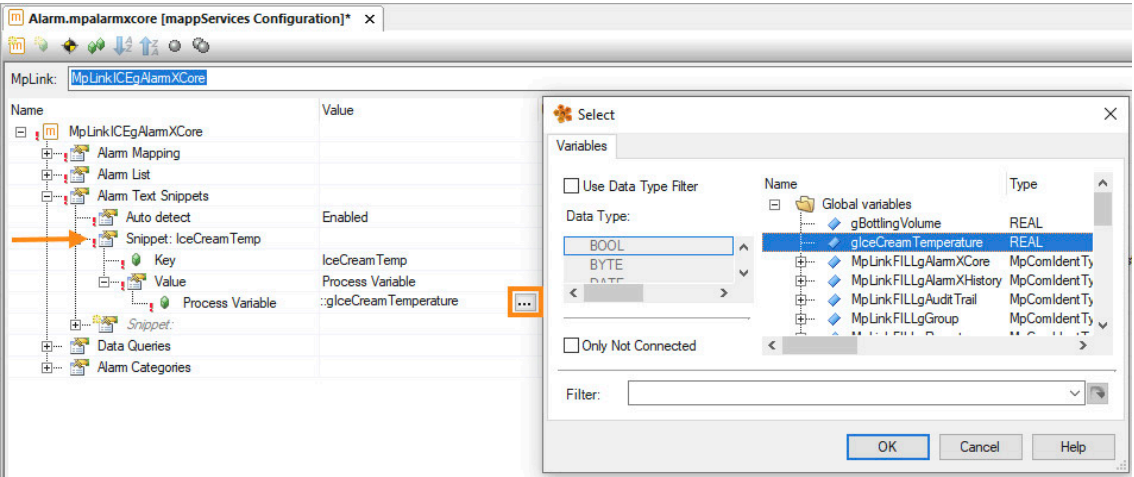


Figure 15: Creating an alarm snippet with the key "IceCreamTemp".

In the next step, the alarm snippet is integrated into the alarm text.

Alarm text snippets can be integrated into the alarm text regardless of whether the text is retrieved from the alarm configuration or imported from the text system.

The syntax is as follows: `{&Key}`

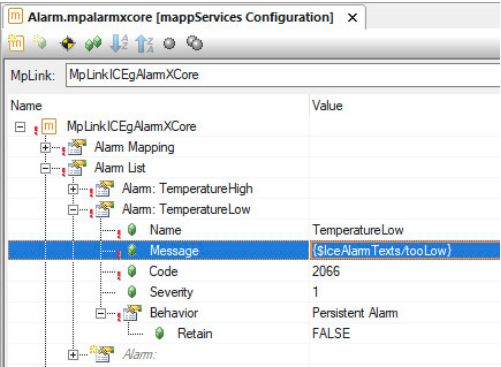


Figure 16: Integrating alarm text snippets into alarm text

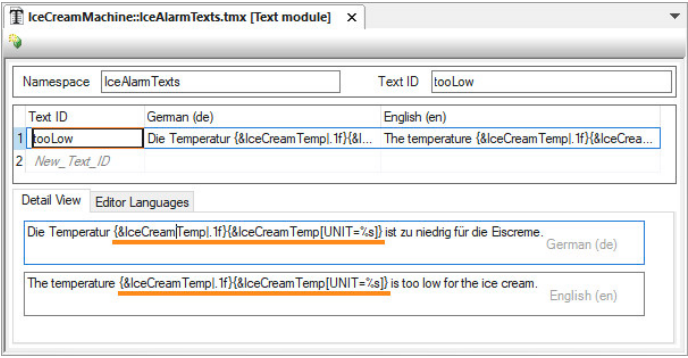


Figure 17: Using the text snippet in the text system

Creating a text system for alarm texts

Exercise: Create a text system for alarm texts

The goal of this exercise is to create a text system and to store alarm texts in German and English. An alarm text is created here that indicates if the temperature of the ice cream is too low.

- 1) Add file "Project languages" in the Logical View.
 - en - English
 - de - German
- 2) In folder "IceCreamMachine" add file "Localizable Texts" and name it "IceAlarmTexts".
 - Namespace - IceAlarmTexts
 - Text ID - tooLow
 - German (de) - Die Temperatur {&IceCreamTemp|.1f|}{&IceCreamTemp[UNIT=%s]} ist zu niedrig für die Eis-creme.
 - English (en) - The temperature {&IceCreamTemp|.1f|}{&IceCreamTemp[UNIT=%s]} is too low for the ice cream.
- 3) Under "TextSystem", add the file "textconfig".
 - System language - en
 - Fallback language - en
 - Target language 1 - en
 - Target language 2 - de
 - Tmxfile1 - IceCreamMachine.IceAlarmTexts.tmx
- 4) Adjust the alarm "Alarm: TemperatureLow" in the MpAlarmXCore configuration.
 - Message - {\${IceAlarmTexts/tooLow}}
- 5) Adjust the "Alarm Text Snippets" in the MpAlarmXCore configuration.
 - Key - IceCreamTemp
 - Value - Process Variable
 - Process Variable - glceCreamTemperature



If the variable glceCreamTemperature falls below the value -20, then the text "The temperature -22.0°C is too low for the icecream." is displayed, for example.

The alarm texts are created in the text file. Additional alarm texts can be added.

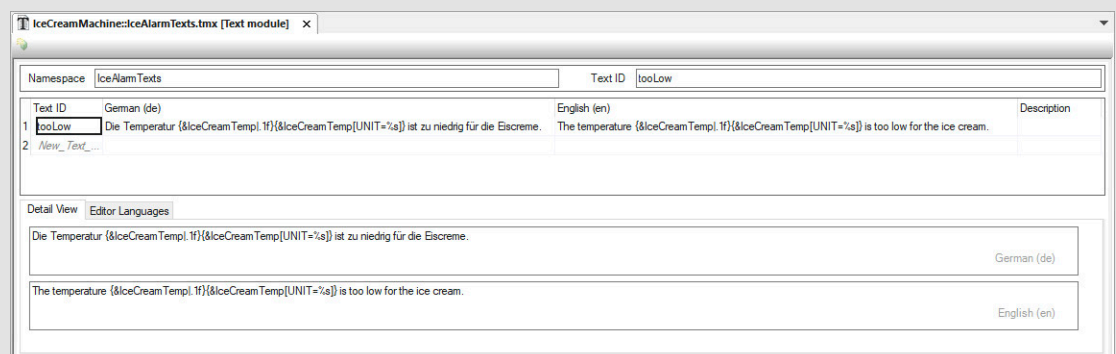


Figure 18: Configured alarm texts of the ice cream machine.



Breaks in the text file are replaced with corresponding ASCII characters when displaying the texts.

Communicating with alarm management via the user interface

Exercise: Communicate with alarm management via the user interface

The goal of this exercise is to communicate with alarm management via the user interface. The user interface is operated via the Watch window.

First the UI function block and associated interfaces are added.

1) Create the following variables in "Variables.var":

```
(*UI-Connection*)
VAR
MpAlarmXListUI_0 : MpAlarmXListUI;
(*MpAlarmXListUI for UI Connection to the ice cream machine*)
UIConnectCore : MpAlarmXListUIConnectType;
(*UI Connection - contains the parameters needed for the connection*)
END_VAR
```

The function blocks are then added and configured. Test the program in the Watch window.

1) Use the function block "MpAlarmXListUI_0" in the initialization subroutine:

- Enable - True
- MpLink - ADR(MpLinkICEgAlarmXCore)
- UIConnect - ADR(UIConnectCore)

2) Call the function block in the cyclic program section:

- MpAlarmXListUI_0()

3) Use and call the function block "MpAlarmXListUI_0" in the exit program:

- Enable - False
- MpAlarmXListUI_0()

Now communication is possible via the user interface.

- 1) Search for 2066 or 2056 in UIConnect under "AlarmList \ Codes".
- 2) Remember the corresponding index.
- 3) Enter the index in "Selectindex".
- 4) Set "Acknowledge" to "True".



Setting "Acknowledge", acknowledges the alarm. The respective alarm is therefore no longer included in the number of pending alarms.

Logging and exporting the alarm history via MpAlarmXHistory

Exercise: Log and export the alarm history via MpAlarmXHistory.

The goal of this exercise is to log and export the alarms that have occurred. The function block "MpAlarmXHistory" is used.

First the mapp AlarmXHistory component is added and edited.

- 1) Under "mapp Services" \ "IceCreamMachine", add the file "MpAlarmXHistory" and name it "IceAlaHis".
- 2) Rename MpLink to "MpLinkICEgAlarmXHistory".

The function block is then added and configured. Test the program in the Watch window.

- 1) Create the following variable in "Variables.var":

VAR

```
AlarmLanguage : STRING[20] := 'de|metric';
(*String which defines language and unit for display and export*)
```

END_VAR

- 2) Use the function block "MpAlarmXHistory_0" in the initialization subroutine:
 - Enable - True
 - MpLink - ADR(MpLinkICEgAlarmXHistory)
 - DeviceName - ADR('CF')
- 3) Call the function block in the cyclic program section, and connect the input "Language" with the string "Alarm-Language".
- 4) Use and call the function block in the exit program.
- 5) Enable monitor mode and test the program. Display MpAlarmXHistory_0 and AlarmLanguage in the Watch window.
 - Customize the "AlarmLanguage" string, e.g. 'en|imperial-us'.
 - MpAlarmXHistory_0 - Export - True
 - Open the file created in the target location.



The alarm history is placed in the configured storage location and can be opened.

8 Diagnostics via WebXs

WebXs provides a web-based connection to mapp. It can be used for diagnostics and to configure mapp components. Components are only displayed in the WebXs if it is active on the controller ("Active = TRUE")

The WebXs is enabled by assigning the library MpWebXs to the software configuration. After the project is transferred, the diagnostic and configuration contents of the respective mapp component are displayed in the web browser.



The WebXs configuration page can be opened from the address bar (**IPAddress/mapp/config**). If System Diagnostics Manager is active, the WebXs can be accessed via "Application status" on the home screen.

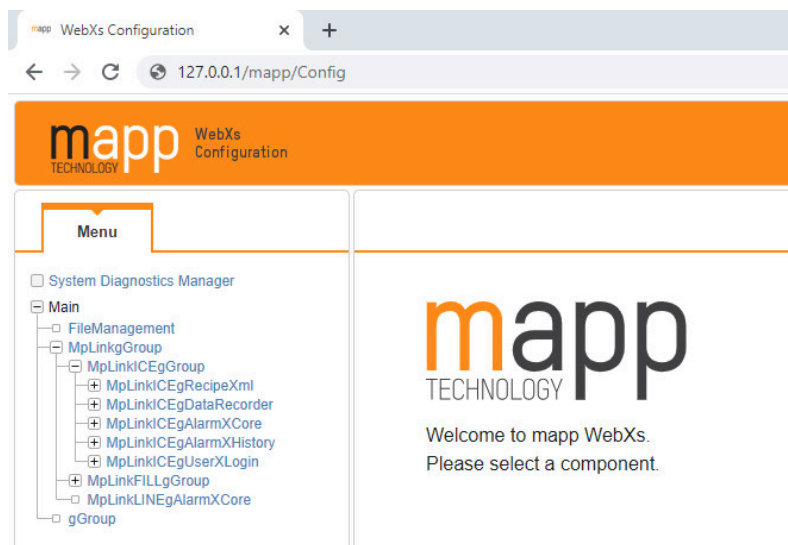


Figure 19: Web diagnostics of a mapp component

The left side shows all components that are active in the system. Clicking on a component shows it as a function block with all of its inputs and outputs. In addition, the current value for each input/output is displayed. Structures (e.g. parameters) can be opened and closed. Component-specific settings can also be made. For information about which configurations can be made for the components, see the description of the individual components.

Diagnostics via WebXs (Web Access)

Exercise: Diagnostics via WebXs (Web Access)

The goal of this exercise is to set up the diagnostics for the mapp components using WebXs (Web Access). After the project is transferred, the diagnostic and configuration contents of the respective mapp component are displayed in the web browser.

- 1) Under "mapp Services", add the file "MpWebXs" and name it "WebXs".
- 2) Transfer the project and restart the controller.
- 3) Call WebXs (Web Access):
 - System Diagnostics Manager - Application status
 - Browser - <http://ip-address/mapp>
- 4) Check the components.

9 Hierarchy with mapp Com - Part I

mapp Com represents the heart of the mapp components and can be used to establish a connection and to structure the components.

 [Services \ mapp Services \ mapp Com: mapp management \ Concept](#)

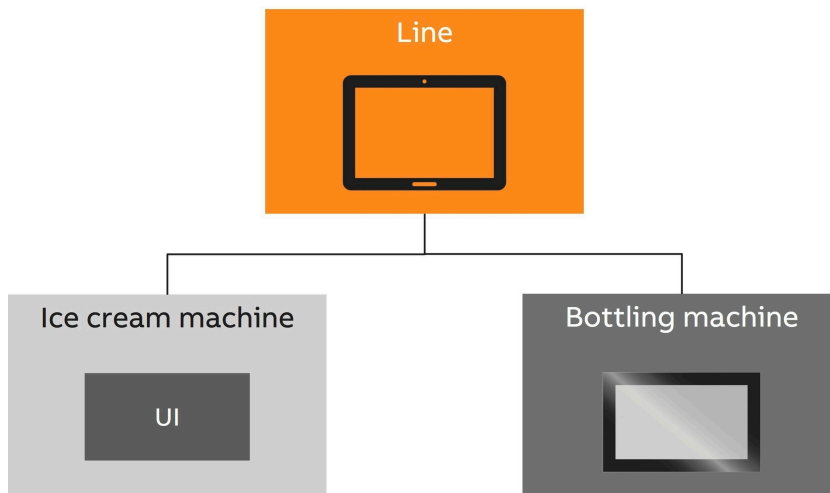


Figure 20: Hierarchy in the project

mapp hierarchy - Part 1

Exercise: Set up the mapp hierarchy - Part 1

The goal of this exercise is to set up a complete machine. In order to do this, the first machine part (the ice cream machine) is completed as a machine group.

- 1) Under "mapp Services" \ "IceCreamMachine", add the file "MpComGroup" and name it "IceGrp1".
- 2) Rename MpLink to "MpLinkICEGroup".
- 3) All MpLinks of the components are specified under "Child components", which are located in the machine group of the ice cream machine.
- 4) The "MpAlarmX" option is selected under "Alarms".
- 5) Optional: Call WebXs (Web Access).

10 Connecting to an HMI application: User interface vs. mapp View

mapp Technology components are represented by function blocks and completely separated from the HMI application. If mapp View is used as the HMI, many mapp Services components can be easily linked to widgets. All necessary data for the HMI application is then loaded from the mapp component according to the language without any further application effort.

When using Visual Components VC4 or third-party HMI applications, mapp offers the option to prepare all relevant data of the mapp Services component (texts, navigation elements, filters, etc.) as process variables in the program and to link them to the HMI application. For this purpose, several components contain UI function blocks that provide all data in a separate structure.

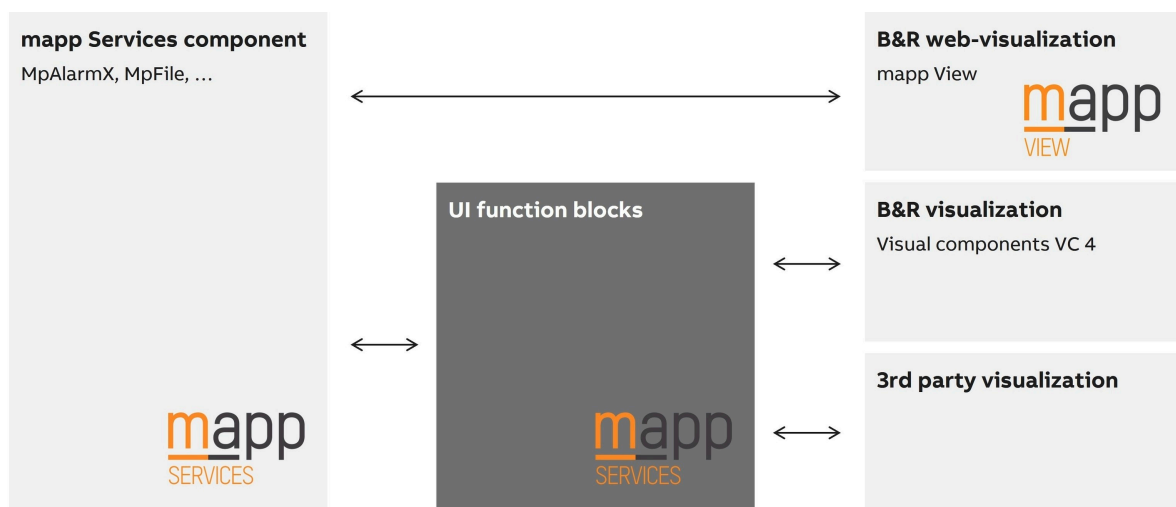


Figure 21: mapp Services UI function block

The UI function blocks can be connected to the corresponding mapp Services component via the MpLink. Each mapp Services component provides the type of a structure that contains all data relevant for the HMI application. This structure must then be instantiated and transferred to the UI function block. Afterwards, the data of the structure can be linked either to a VC4-based HMI application or any other third-party HMI application.

Binding with mapp View

Some widgets communicate with mapp function blocks; for example, widget "AlarmList" communicates with function block mapp AlarmX.

For this communication to take place, library **MpServer** must be transferred to the target system. "MpServer" is used for internal communication.



Which widgets communicate with mapp function blocks?

[Visualization \ mapp View \ Widgets \ Additional information \ mapp communication](#)

11 Event management with mapp Audit

mapp Audit can log different events. These can originate from the HMI application, MpUserX or user-defined events. The user can determine the format in which the events are stored in a file.

One entry is created for each event. The entry is managed by MpAudit and stored in memory. The associated text for the event can come from the Automation Studio text system. The namespace and text ID are used for linking to the text system.

The function block **MpAuditTrail** manages event logging.



[Services \ mapp Services \ mapp Audit: Event management \ Concept](#)

Creating an event management system with MpAuditTrail

Exercise: Prepare the event management system with unit and text

To demonstrate the wide range of functions of MpAuditTrail, a few configurations are made in advance. A global variable is created that stores the fill quantity of the ice cartons to be filled. A unit is defined for the variable. In addition, a text system is created to help the user understand the audit output.

- 1) Create the following variable in "Global.var":

```
(*Bottling machine*)
VAR
gBottlingVolume : REAL := 125;
(*Current volume for the bottling machine*)
END_VAR
```

- 2) Open "OpcUaMap.uad"; under "Global variables", right click on "gBottlingVolume" and click on "Enable tag".
- 3) Search for "milliliter" in the toolbox and drag and drop the entry with the symbol "ml" onto the tag "gBottlingVolume".
- 4) Under "mapp Services", open the file "Unit" and under "Unit conversion" \ "Units" \ "Unit 2" \ "Engineering unit", enter "MLT" in the field "Unitcode".
 - Under "Unit 2" \ "Display units" \ "Metric", enter "MLT" in the "Unit code" field.
 - Under "Unit 2" \ "Display units" \ Imperial", enter "OZI" in the "Unit code" field.
 - Under "Unit 2" \ "Display Units" \ ImperialUS", enter "OZA" in the "Unit code" field.
- 5) Add the file "Localizable Texts" in the "BottlingMachine" folder and name it "BottAuditTexts".
 - Namespace - BottAuditTexts
 - Text ID - Root
 - German (de) and English (en) - {&evtime[TIME=%u%R]}={\$BottAuditTexts/{&ev}}
 - Text ID - 16
 - German (de) -
 - Wertänderung: alter Wert = {&old|.2f}{&old[UNIT=%s]}, neuer Wert = {&new|.2f}{&new[UNIT=%s]}
 - English (en) -
 - Value changed: old value = {&old|.2f}{&old[UNIT=%s]}, new value = {&new|.2f}{&new[UNIT=%s]}



The spelling in the localizable text files must be checked in order to ensure that all texts are displayed correctly. Soft and hard **line breaks** are not permitted to be included!

- 6) Open "TC.textconfig" under "Tmx files for target" and add the file "BottlingMachine.BottAuditTexts.tmx".

Exercise: Create an event management system with MpAuditTrail

The goal of this exercise is to create a functional event management system with function block "MpAuditTrail". Every change to the value of the filling volume of the variable "gBottlingVolume" is recorded and stored. At the end, the event list is exported to the configured storage location.

First, the mapp Audit component is added and edited. Next, the program is created.

- 1) Under "mapp Services" \ "BottlingMachine", add the file "MpAuditTrail" and name it "BottlAudit".
- 2) Rename MpLink to "MpLinkBOTTgAuditTrail".
- 3) Select the option "TextSystem" under "Audit" \ "Text source".
- 4) Enter "BottAuditTexts/Root" under "Audit" \ "Text source" \ "Format text source".
- 5) Select the global variable "gBottlingVolume" under "Variable monitor" in the field "PV name".
- 6) Under "Export" in the "Encrypt" field, select option "False".
- 7) Add "ST Program All In One" and name it "Bott_Audit".
- 8) Create the following variable in "Variables.var":

```
VAR
AuditLanguage : STRING[20] := 'de|metric';
(*String which defines language and unit for display and export*)
END_VAR
```

The function block is then added and configured. Test the program in the Watch window.

- 1) Use the function block "MpAuditTrail_0" in the initialization subroutine (Enable, MpLink, DeviceName). Also call the variable "gBottlingVolume".
- 2) Call the function block in the cyclic program section, and connect the input Language with the string Audit-Language.
- 3) Use and call the function block "MpAuditTrail_0" in the exit program.
- 4) Enable monitor mode and test the program. Display MpAuditTrail_0 and gBottlingVolume in the Watch window.
 - Change the value of gBottlingVolume.
 - MpAuditTrail_0 - Export - True



The event list is stored at the configured location by switching the "Export" input and can be opened. The language/unit specified at the "Language" input is also used in the "Audit" file.

```
Audit-Trail Export File
Information:
Export=1593068924.499 +02: by Tom
TextSource=AR/TextSystem | (V1.00)
Language=de
Data:
07:08 Wertänderung: alter Wert = 130.00ml, neuer Wert = 120.00ml
```

Figure 22: Exported event list with 'de|metric'

```
Audit-Trail Export File
Information:
Export=1593069037.489 +02: by Tom
TextSource=AR/TextSystem | (V1.00)
Language=en
Data:
07:08 Value changed: old value = 4.40floz[US], new value = 4.06floz[US]
|
```

Figure 23: Exported event list with 'en|imperial-us'

Expanding the event management system by adding MpAuditTrail - Recording additional events

Exercise: Expand the event management system - Record additional events

In the library "MpAudit", TMX files that have already been created are provided and serve as reference. All possible events are already covered.

The goal of this exercise is to use the supplied text files for orientation and to implement them. The added TMX file must also be specified in the text system.

1) Add the file "Localizable Texts" in the "BottlingMachine" folder and name it "**BottAuditSys**".

- Namespace - BottAudit/Sys
- Text ID - Root
- German (de) - {&evtime[TIME=%u%R]} {&op} {=\$BottAudit/Sys/{&ev}}
- English (en) - {&evtime[TIME=%u%R]} {&op} {=\$BottAudit/Sys/{&ev}}
- Text ID - 2
- German (de) - {=\$BottAudit/Sys/UserX/{&act}}
- English (en) - {=\$BottAudit/Sys/UserX/{&act}}
- Text ID - 16
- German (de) - Wertänderung: alter Wert = {&old|.2f}{&old[UNIT=%s]}, neuer Wert = {&new|.2f}{&new[UNIT=%s]}
- English (en) - Value changed: old value = {&old|.2f}{&old[UNIT=%s]}, new value = {&new|.2f}{&new[UNIT=%s]}
- Text ID - 33
- German (de) - Alarm '{&name}' Status geändert: {&stold} zu {&stnew}
- English (en) - Alarm '{&name}' changed state: {&stold} to {&stnew}
- Text ID - 48
- German (de) - {=\$BottAudit/Sys/Recipe/{&act}}
- English (en) - {=\$BottAudit/Sys/Recipe/{&act}}

2) Add the file "Localizable Texts" in the "BottlingMachine" folder and name it "**BottAuditUserX**".

- Namespace - BottAudit/Sys/UserX
 - Copy entries from the "MpAudit" library located in the "TxtUserF.tmx" file.
- 3) Add the file "Localizable Texts" in the "BottlingMachine" folder and name it "**BottAuditRec**".
 - Namespace - BottAudit/Sys/Recipe
 - Copy entries from the "MpAudit" library located in the "TxtRecipeF.tmx" file.
 - 4) Open "TC.textconfig" under "Tmx files for target" and add the individual text files.
 - 5) Enter BottAudit/Sys/Root in the mapp Audit component under "Audit" \ "Text source" \ "Format text source".
 - 6) In the mapp Audit component, click on "Change advanced parameters visibility".
 - 7) Select the events "MpRecipe events" and "MpAlarmX Audit events" under "Audit" \ "Events".
 - 8) Select the option "MpAudit" in the mapp Recipe configuration under "Recipe" \ "Auditing".
 - 9) Select the option "MpAudit" in the mapp AlarmX history configuration under "Alarm history" \ "Auditing".

Communicating with the event management system via mapp View

Exercise: Communicate with the event management system via mapp View

The goal of this exercise is to communicate with the alarm management system via mapp View. A new HMI page is created for this and the "AuditList" widget is placed on it. The value changes of the fill volume of the variable "gBottlingVolume" are displayed in this widget.

- 1) Add the "AuditList" widget to "ContentAudit".



To use this widget, the "MpServer" library must be transferred to the controller.

- 2) Click on the 3 points under "Data" \ "mpLink" \ in the line "Binding".
 Select the MpLink "MpLinkBOTTgAuditTrail" in the "mapp" tab.
 Select the "Read / Write" option under "Binding Mode".
 Enable the "ContentRelated" checkbox under "Bindings Set Id".
- 3) Add the AuditListItems "timestamp" and "text" in the "AuditList" widget.
- 4) Optional: Check the binding in the "BottVisu.vis" file.



The language that the audit texts are displayed in can be determined using the "LanguageSelector" widget.

Date and time	Audit message
Tuesday, August 04, 2020 2:17:19 PM	12:17 Value changed: old value = 125.00ml, new value = 200.00ml
Tuesday, August 04, 2020 1:59:09 PM	11:59 Value changed: old value = 125.00ml, new value = 1250.00ml
Monday, August 03, 2020 2:04:53 PM	12:04 Value changed: old value = 500.00ml, new value = 1500.00ml
Monday, August 03, 2020 2:04:49 PM	12:04 Value changed: old value = 125.00ml, new value = 500.00ml

Previous 1 Next

Figure 24: "AuditList" widget with the entries of the value change of variable "gBottlingVolume".

12 User management with mapp UserX

mapp UserX sets up user management. Roles and users are created using the user role system in Automation Studio (OPC UA compliance included) and then managed using MpUserX. This includes access rights, user data, password definition (letters, numbers, uppercase/lowercase, etc.), logging in/out and connecting to the HMI application.

If a user logs in via mapp UserX, a login session is opened. This can be started via the application ("**Application Session**") or via the HMI application ("**Visualization Session**"). This session can be used for user functions such as logging in and out or changing the password.

In this training module, the user role system is used in combination with mapp UserX and a visualization session.



[Services \ mapp Services \ mapp UserX: User management with the user role system \ Concept Programming \ Access & Security \ User role system \ Configuration \ Automation Studio configuration](#)

Access & Security - User management system

Automation Studio supports configuration of a user system, a role system and a certificate management system as well as management of SSL configurations. These configurations are managed in the "Access & Security" package.

User role system

Two configuration elements are stored in the Configuration View in the folder "Access & Security".

Basic roles for the system can be created in the "Role" file. These can be an operator, a service technician or an administrator, for example.

In the "User" file, individual users can then be created to which one or more roles can be assigned.

Users as well as the associated roles can be adapted at runtime.

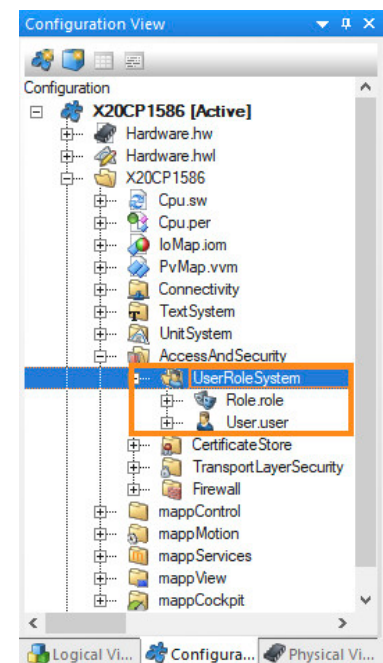


Figure 25: Configuration package "UserRoleSystem"

Creating a user management system with MpUserX

Exercise: Configuring the user role system

The user role system is filled with data so that users and roles can be used.

- 1) Add two new roles under "AccessAndSecurity" \ "UserRoleSystem" \ "Role":
 - Operator - Role ID: 3
 - Service - Role ID: 4
- 2) Add three new users under "AccessAndSecurity" \ "UserRoleSystem" \ "User":
 - Jane - Password: Jane - Assigned Role: Service
 - Dave - Password: Dave - Assigned Role: Operator
 - Tom - Password: Tom - Assigned Role: Administrator



The users and roles are created in the system and can be used in the project.

Name	User ID	Password	Roles
Anonymous	1		Assigned Role [1] Everyone Assigned Role [2]
Jane	2	*****	Assigned Role [1] Service Assigned Role [2]
Dave	3	*****	Assigned Role [1] Operator Assigned Role [2]
Tom	4	*****	Assigned Role [1] Administrators Assigned Role [2]

Name	Role ID	Description
Administrators	1	Role for..
Everyone	2	Role for..
Operator	3	Role for..
Service	4	Role for..

Figure 26: Configuration of the users and roles.

Exercise: Edit the mapp UserX configuration

The goal of this exercise is to modify the mapp UserX configuration so that password policies apply to all users. The preferred language and unit can also be defined for each user.

- 1) Under "mapp Services" \ "BottlingMachine", add the file "MpUserX" and name it "BottlUserX".
- 2) Enter the password length (e.g. "3") under "General settings" \ "Password length".
- 3) Select the option "True" under "General Settings" \ "Edit users with same user-level".
- 4) Under "Users" in the "Name" field, select each name and enter the corresponding data.
- 5) Under "Roles" in the "Name" field, select each role and enter the corresponding data.



Different levels are defined for the roles. It is important that property "Administrator = TRUE" is defined for the administrator role.



A user can be uniquely identified in the system via a user management system. Identification takes place by entering a username and password. This ensures protection against external access and thus the confidentiality of the contents in the respective system. In addition, different functions can be unlocked or locked in the system depending on the user.

Once logged in, a user is registered in the system and rights are assigned to their role. For example, when Jane logs in, she can only access content that has been approved for the service technician. A user is therefore always bound to a role.

Exercise: Communicate with the user management system via mapp View

The goal of this exercise is to communicate with the user management system via mapp View. A new HMI page is created for this and the "UserList" widget is placed on it. Three buttons are also added for different functions (to add, delete and modify users).

- 1) Select the option "MpUserX" under "Server configuration \ "Authentication mode" in the mapp View configuration.
- 2) In the mapp View configuration, set "Server configuration" \ "Authentication mode" \ Set "User preferences" to True.
- 3) Add the following widgets to "ContentUser":
 - Login widget
 - LogoutButton widget
 - Label widget
 - LoginInfo widget
 - 3x Button widget
 - UserList widget



To use this widget, the "MpServer" library must be transferred to the controller.

- 4) Add the following UserListItem to the UserList widget: UserName, FullName, Roles, LastLogin, IsAdmin.
- 5) Configure the three button widgets: Delete user, Add user, Modify user.



[Visualization \ mapp View \ Widgets \ Data \ UserList \ Concept \ Working with widget UserList](#)

- 6) Optional: Check the event binding in the "BottVisu.vis" file.

Test the configuration in the HMI application:

- 1) Log in users with the role "Administrator" via the login widget.



To edit a user, the logged-in user must be in an mpUserX administrator role. With the configuration "Edit users with same user level" = True, the Administrator is permitted to edit other users with a lower or the same user level.

- 2) Check the UserList widget and add new user.



The UserList widget can be used to display the available user information. In addition, a user can be added, edited or deleted via different actions.

Username: Tom
Password: [Redacted]
Login Logout

Logged in as: Tom

User changed to: Tom

Name	Full name	Roles	Last login	Admin
Anonymous		Everyone	Thursday, April 29, 2021 11:02:42 AM	
Dave	Dave Miller	Operator	Thursday, April 29, 2021 11:02:42 AM	
Jane	Jane Huber	Service	Thursday, April 29, 2021 11:02:42 AM	
Tom	Tom Meier	Administrators	Thursday, April 29, 2021 11:05:33 AM	

Previous 1 Next

Add user Remove user Modify user

Figure 27: View of the page "UserPage" with the UserList widget.

Add a user

User: Username
Name: Full name
Password: Password
Confirm: Confirm password

User account requirements:

Available roles:

- ☐ Administrators
- ☐ Everyone
- ☐ Operator
- ☐ Service

OK Cancel

Figure 28: Dialog box that appears when you add a user via the visualization session, e.g. via the "Add user" button.

13 File management system with mapp File

mapp File provides a file management system as well as a connection to the HMI application to display files. The MpFileManager configuration is used in combination with mapp View.



[Services \ mapp Services \ mapp File: File management system \ Concept](#)

Creating the file management system with mapp File

Exercise: Create the file management system with mapp File

The goal of this exercise is to open a Recipe file, modify it and save it. This is done using the mapp File component, the "TextPad" widget and two buttons.

First the mapp File component is added.

- 1) Under "mapp Services" \ "BottlingMachine", add the file "MpFileManager" and name it "BottlFile".
- 2) Select the "Enabled" option under "File manager" \ "USB detection".
- 3) Enter "CF" in the "Device name" field under "File manager" \ "Devices".
- 4) Enter "Temp Folder" in the "Text" field under "File manager" \ "Devices".
- 5) Under "File manager" \ "Access", select "All" in the "Folder" field; under "Roles", select "Everyone". Set all underlying rights to "True".

The mapp View connection is then configured.

- 1) Add the "TextPad" widget to "ContentFile".



To use this widget, the "MpServer" library must be transferred to the controller.

- 2) Add two buttons:
 - ° Save - Click event
 - ° Open - Click event
- 3) Configure the events; see:



[Services \ mapp Services \ mapp File: File management system \ Getting started \ Creating a text editor for CNC programs in mapp View \ Adding and configuring a widget](#)

- 4) Optional: Check the event binding in the "BottVisu.vis" file.

Finally, the configuration is tested via mapp View.

- 1) Open the recipe file.
- 2) Overwrite the individual values.
- 3) Save the recipe file.



The recipe, which is located in the configured storage location, can be opened and edited. If these values are overwritten and saved, the recipe can be loaded into the project with "MpRecipeXml - Load". This overwrites the parameters in structure "IceFormula".

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <DATA>
3   <Element Name="Ice_Recipe:IceFormula" Type="FvParameter">
4     <Group ID="Ice_Recipe:IceFormula">
5       <Property ID="Milk" DataType="REAL" Value="205" />
6       <Property ID="Cream" DataType="REAL" Value="250" />
7       <Property ID="Sugar" DataType="REAL" Value="60" />
8       <Property ID="EggYolk" DataType="USINT" Value="3" />
9       <Property ID="Flavour" DataType="STRING" Value="Chocolate" />
10    </Group>
11  </Element>
12 </DATA>
13

```

Open

Save

Figure 29: The ice cream recipe is opened in the "TextPad" widget.

USB flash drive recognition can be used when working with a real controller (without ArSim). Inserted USB flash drives are automatically detected if the "Enabled" option is selected for the "USB detection" function in the mapp file configuration. The flash drives are displayed in the file browser of a mapp View HMI application and can be used.

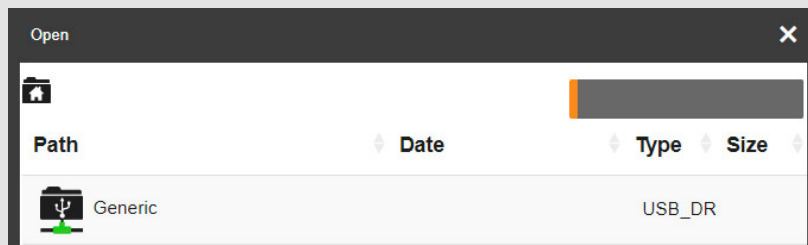


Figure 30: The B&R USB flash drive inserted at runtime is recognized and can be used.

14 Creating reports with mapp Report

mapp Report creates reports that can be exported to a storage medium.

The entire content of a report is defined in the MpReportCore configuration. A report consists of a header and the main content. Text in the report can come from the Automation Studio text system. The namespace and text ID are used for linking to the text system.

A report can be created using the function block **MpReportCore**.



[Services \ mapp Services \ mapp Report: Creating reports \ Concept](#)

Creating a report with MpReportCore

Exercise: Create a report with MpReportCore.

The goal of this exercise is to create or export a report using the function block MpReportCore. This will export the latest state of the gBottlingVolume and glceCreamTemperature variables as a report at the configured location. First, the mapp Report component is added and edited. Next, the program is created.

- 1) Under "mapp Services" \ "BottlingMachine", add the file "MpReportCore" and name it "BottlRep".
- 2) Rename MpLink to "MpLinkBOTTgReport".
- 3) Configure the component; see:



[Services \ mapp Services \ mapp Report: Creating reports \ Getting started \ Creating a report \ Adding the mapp component](#)

- 4) Under "Tables" \ "Table: TableContent" \ "Row: 1" \ "Column: 1"
 - Row 1 \ Column 1 - Filling volume of the ice cream
 - Row 1 \ Column 2 - Temperature of the ice cream
 - Row 2 \ Column 1 - gBottlingVolume
 - Row 2 \ Column 2 - glceCreamTemperatures
- 5) Add "ST Program All In One" and name it "Bott_Rep".
- 6) Create the following variable in "Variables.var":


```
VAR
ReportLanguage : STRING[20] := 'en';
(*String which defines language for display and export*)
END_VAR
```

The function block is then added and configured. Test the program in the Watch window.

- 1) Use the function block "MpReportCore_0" in the initialization subroutine (enable, MpLink, name, DeviceName).
- 2) Call the function block in the cyclic program section, and connect the input Language with the string ReportLanguage.
- 3) Use and call the function block "MpReportCore_0" in the exit program.
- 4) Enable monitor mode and test the program.
 - MpReportCore_0 - Generate - True



The report is stored at the configured location by switching the "Generate" input and can be opened. The language specified at the language input is also used in the report.

Production Report
2020-07-01 08:26:18

Filling volume of the ice cream	Temperature of the ice cream
125.000000 ml	-8.000000 °C

Figure 31: Saved report stored in the configured storage location.

15 Alarm management with mapp AlarmX

- Part II

The alarm system runs on the controller on a standalone basis. The mapp View "AlarmList" widget facilitates the process of creating an HMI application for alarms.

Features of the alarm system:

- Collecting mapp alarms and user alarms
- Recording active and historic alarm states
- Managing alarm texts and properties via the mapp configuration
- Binding the alarm system and text system via the configuration
- Consolidating different alarms based on "reactions"
- Exporting historic alarms to the file system
- Binding the alarm system to the "AlarmList" widget via mapp Link binding quickly and easily

The behavior of an alarm can be defined in such a way that the alarms are triggered directly by MpAlarmX based on conditions that can be defined in the configuration.

The alarm handling can send an alarm as soon as a definable temperature variable has exceeded a limit temperature defined in the configuration, for example.

Alternatively, the behavior can be set so that the alarm must be set from the application via the MpAlarmXSet function.



[Services \ mapp Services \ mapp AlarmX: Alarm management \ Concept](#)

- Alarm properties
- Alarm monitoring
- Alarm mapping

15.1 Severity

The severity indicates the severity "level" of an alarm. This is a freely selectable integer that can be defined as needed. For example, non-critical alarms can be assigned a severity of 10, while critical alarms can be given a severity of 100. The severity level can be used later for filtering alarms or for monitoring alarms with the same severity level.

15.2 Alarm behavior \ Alarm monitoring

In addition to displaying persistent alarms and edge alarms, monitoring functions are also provided for process variables. This enables the monitoring of automatic limit values for process variables and, if these limits are exceeded, the automatic triggering of alarms. The alarm management is defined in the configuration. A brief overview of the types of alarm monitoring will follow.

Level monitoring alarm

This alarm monitors the level of a process variable (PV). The PV is defined in the configuration. It is possible to define two lower limit values and two upper limit values here. If an upper or lower limit value is exceeded, the alarm is triggered.

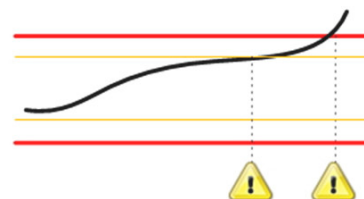


Figure 32: Level monitoring alarm

Deviation monitoring alarm

This alarm monitors deviation from a defined level. In this case, a PV is specified with the current value and a second PV defines the setpoint. If the current value deviates from the setpoint by a defined tolerance, an alarm is generated.

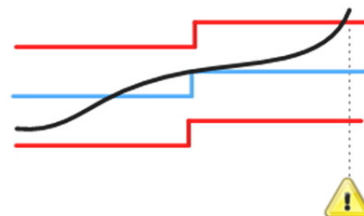


Figure 33: Deviation monitoring alarm

Rate of change monitoring

This alarm monitors the rate of change of a PV. The PV is defined in the configuration. It is possible to define two lower limit values and two upper limit values here. If an upper or lower limit value is exceeded, the alarm is triggered.



Figure 34: Rate-of-change monitoring alarm

Discrete value monitoring alarm

This alarm monitors certain PV values. The PV is defined in the configuration. If a PV changes to a specific value, an alarm is triggered.



Figure 35: Discrete value monitoring alarm

15.3 Mapping alarms

In addition to alarm properties, users can also define how an alarm should behave in the complete system. They can decide what should happen for a particular alarm, or what actions should be carried out for a certain trigger. This is done using alarm mapping.

In each case, a trigger is connected to one or more actions (reactions). A trigger can be a specific alarm or a severity level.

The following diagram illustrates the relationship between the different alarms. These relationships can be assigned in the alarm configuration directly, or assigned to a specific reaction via the severity level. For example, the "NoProduct", "InvalidProduct" and "SawBattered" alarms will always trigger a "Stop" reaction. Using reactions significantly reduces the number of states to be analyzed in the application.

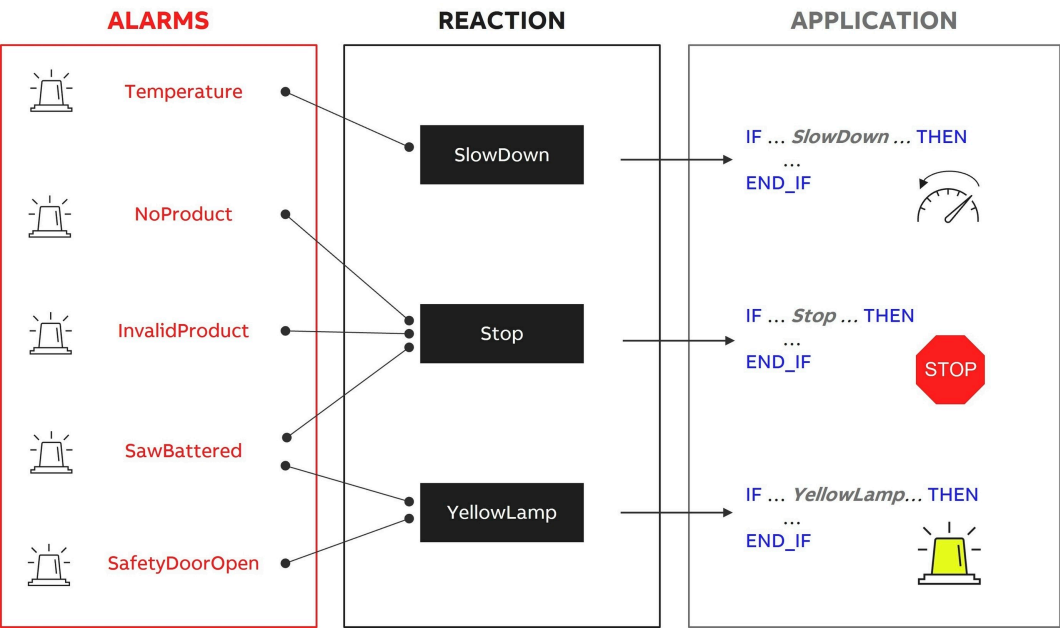


Figure 36: Relationship between alarms, severity and the application program

Alarm mapping with alarm names

Whenever a specific alarm is triggered, one or more actions are carried out. The figure shows that the "ConfirmLamp" reaction is triggered when the ChangeBottling alarm occurs and the above defined limit value is exceeded at the same time.

A number of different alarms can initiate the same reaction.

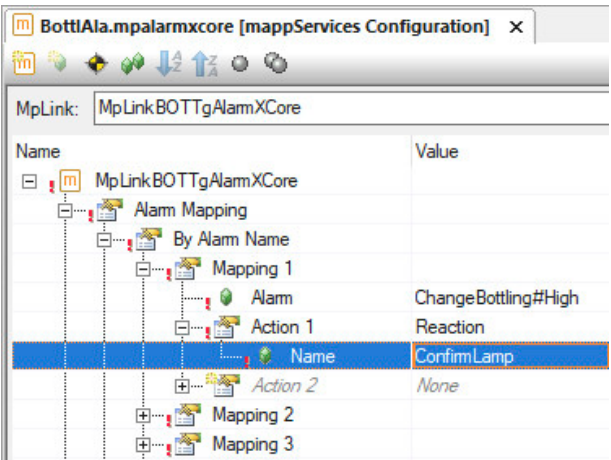


Figure 37: "ConfirmLamp" reaction triggered by "TempHigh" alarm

Alarm mapping with severity

The alarm configuration can be used to set reactions for specific severities. Alarms that have the same severity or a severity within a given value range can trigger a specific reaction, for example.

As shown in the figure, a value range can also be entered for the severity.

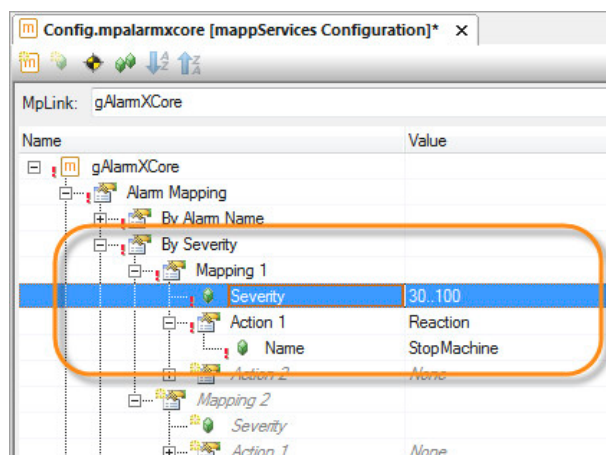


Figure 38: Alarm mapping using a range of severity values

Evaluating reactions

The MpAlarmXCheckReaction function can be used to check whether a certain reaction is active. The application software can then respond to this reaction appropriately.

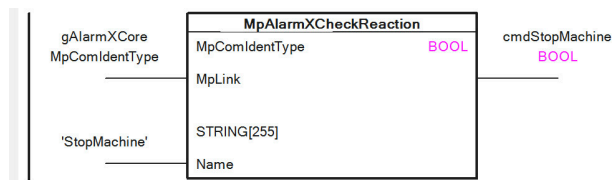


Figure 39: Querying whether the "StopMachine" reaction is active

15.4 Displaying alarms in the HMI application

The "AlarmList" widget is used to display alarms in the HMI application. It is a container widget to which "Alarm-Listitem" widgets are added. "AlarmItem" widgets represent the individual columns of the alarm list.



In the properties for the "AlarmList" widget, the "itemsPerPage" parameter is used to define the number of entries displayed on the list. A mapp binding is used to bind the "AlarmList" widget to the alarm system instance.

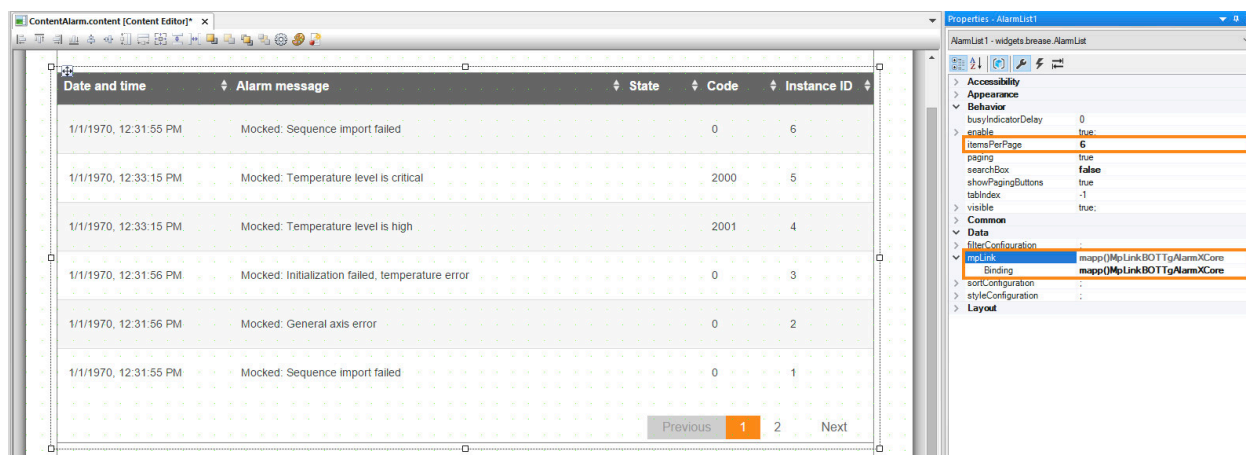


Figure 40: Properties of the "AlarmList" widget – mapp binding and number of list elements

The alarm information that should be displayed is defined in the "AlarmItem" widget. The column width is also defined in the "AlarmItem" widget.

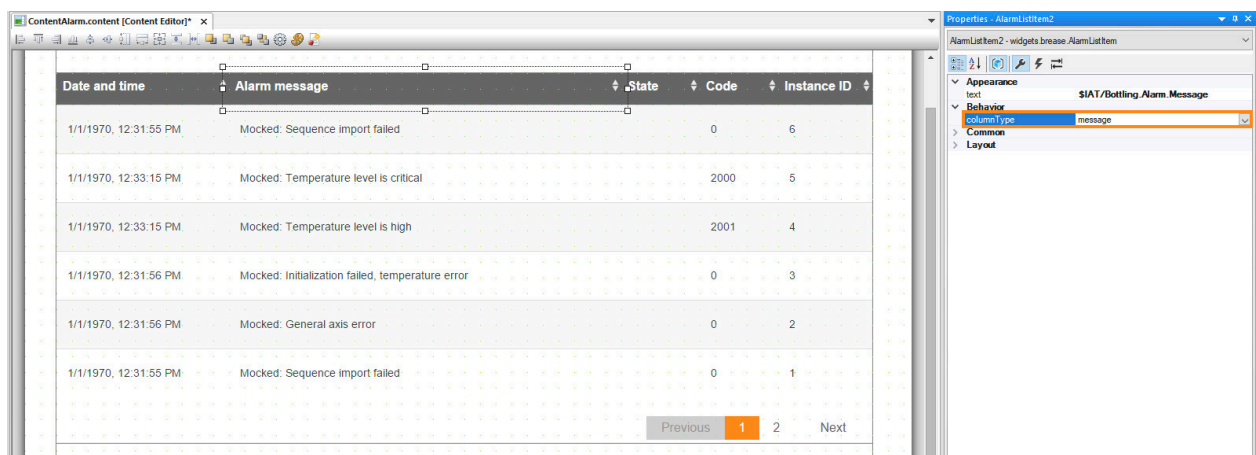


Figure 41: "AlarmItem" widget properties used for displaying message text

Creating an alarm management system with MpAlarmX

Exercise: Create an alarm management system with MpAlarmX

The goal of this exercise is to create an alarm management system for the filling machine. Function block "MpAlarmX-Core" is used at the start. Alarms are then triggered and reset by the user.

The following alarms are configured:

The following alarms are configured:

- BottVolumeChanged - When the fill volume of the filling container is set to a different value.
- ChangeBottling - When the fill volume is greater than 1000 units. Then the message should state that buckets must be used as filling containers.

First, the mapp AlarmX component is added and edited. Next, the program is created.

- 1) Under "mapp Services" \ "BottlingMachine", add the file "MpAlarmXCore" and name it "BottAla".
- 2) Rename MpLink to "MpLinkBOTTgAlarmXCore".
- 3) Under "AlarmList" create an edge alarm with the name "BottVolumeChanged".
- 4) Under "AlarmList" create a level monitoring alarm with the name "ChangeBottling". The upper static limit should be 1000.
- 5) Add an "ST Program All In One" in the "BottlingMachine" folder and name it "Bott_Alarm".

The text system is then used for the alarm texts.

- 1) Name the text file "BottAlarmTexts".
- 2) This text should be displayed for "volumeChanged":
 Das Füllvolumen wurde auf X,XX Einheit geändert.
 The filling volume changed to X.XX unit .
- 3) This text should be displayed for "bottlingBucket":
 Das Füllvolumen ist sehr groß: X,XX Einheit. Eimer verwenden!
 The filling volume is very large: X.XX unit. Use a bucket!

The function block is then added and configured.

- 1) Use the function block "MpAlarmXCore_0" in the initialization subroutine.
- 2) Call the function block in the cyclic program section.

- 3) Come up with logic in the cyclic program part that recognizes whether the filling volume has changed. If this is the case, the configured alarm should be triggered and the logic is reset.
- 4) Use and call the function block "MpAlarmXCore_0" in the exit program.

The program is tested in the Watch window.

- 1) Enable monitor mode and test the program. Display MpAlarmXCore_0 and gBottlingVolume in the Watch window.
 - Change gBottlingVolume.
 - Monitor the status of ActiveAlarms and PendingAlarms

Exercise: Communicate with the alarm management system via mapp View

The goal of this exercise is to communicate with alarm management via mapp View. A new HMI page is created for this and the "AlarmList" widget is placed on it. The value changes of the fill volume of the variable "gBottlingVolume" are displayed in this widget, for example.

- 1) Add the AlarmList widget to "ContentAlarm", see



[Visualization \ mapp View \ Widgets \ Use cases \ Alarm use cases \ Alarms: Displaying alarms in a table](#)



To use this widget, the "MpServer" library must be transferred to the controller.

- Click on the 3 points under "Data" \ "mpLink" \ in the line "Binding".
- Select the MpLink "MpLinkBOTTgAlarmXCore" in the "mapp" tab.
- Select the "Read / Write" option under "Binding Mode".
- Enable the "ContentRelated" checkbox under "Bindings Set Id".

- 2) Optional: Checking the binding and event binding in the "BottVisu.vis" file.

Consolidating alarm states using reactions

Exercise: Consolidate alarm states using reactions

The goal of this exercise is to ensure that a lamp is automatically switched on if the filling volume exceeds 1000 units. As soon as the filling volume falls below this limit, the lamp is switched off again. This project is implemented with alarm mapping or a reaction to the limit value.

- 1) See Automation Help:



[Services \ mapp Services \ mapp AlarmX: Alarm management \ Concept](#)

- Alarm mapping according to alarm name
- Reaction
- Alarm monitoring / Reaction to limit values

- 2) Edit the mapp Alarm configuration.
- 3) Create variable "doLamp" of data type "Bool" to display the status of the lamp.

- 4) Add source code and use function "MpAlarmXCheckReaction".



In the MpAlarmXCore configuration, the setting "Reaction until acknowledged" for the alarm "Change-Bottling" can be changed via the advanced parameters. This setting affects how long "Reaction" is True.

Displaying the alarm history via mapp View

Exercise: Display the alarm history via mapp View

The goal of this exercise is to display the alarm history in mapp View. The "AlarmHistory" widget is placed on the same HMI page as the "AlarmList" widget. The "TabControl" widget is used to ensure that both widgets have enough display area.

- 1) First the mapp AlarmXHistory component is added and edited.
- 2) The function block is then added and configured.
- 3) On the AlarmPage, add the "TabControl" widget with two TabItems.
- 4) Drag the "AlarmList" widget into the workspace of the first TabItem.
- 5) Add the "AlarmHistory" widget to the workspace of the second TabItem.
- 6) Add "AlarmHistoryItems" from types "timestamp", "message", "old state", "new state" and "code".



Both the "AlarmList" widget and the "AlarmHistory" widget can be placed on one HMI page.

Current		History			
Date and time	Alarm message	Old State	New State	Code	
Tuesday, August 04, 2020 2:17:19 PM	The filling volume changed to 200.00ml.			3000	
Tuesday, August 04, 2020 2:14:32 PM	The filling volume changed to 1250.00ml.			3000	
Tuesday, August 04, 2020 1:59:09 PM	The filling volume changed to 1250.00ml.			3000	
Tuesday, August 04, 2020 1:59:09 PM	The filling volume is very large: 1250.00ml. Use a bucket!			3100	
Monday, August 03, 2020 2:04:53 PM	The filling volume changed to 1500.00ml.			3000	
Monday, August 03, 2020 2:04:53 PM	The filling volume is very large: 1500.00ml. Use a bucket!			3100	

Previous
1
2
Next

Figure 42: "TabControl" widget on the top right with a currently open AlarmHistory.

16 Hierarchy with mapp Com - Part II

The hierarchy is created using mapp Com, see [9 "Hierarchy with mapp Com - Part I" on page 28](#).

mapp hierarchy - Part 2

Exercise: Set up the mapp hierarchy - Part 2

The goal of this exercise is to set up a complete machine. For this, the second machine part (the filling machine) is completed as a machine group.

- 1) Under "mapp Services" \ "BottlingMachine", add the file "MpComGroup" and name it "BottlGrp2".
- 2) Rename MpLink to "MpLinkBOTTgGroup".
- 3) All MpLinks of the components are specified under "Child components", which are located in the machine group of the filling machine.
- 4) The "MpAlarmX" option is selected under "Alarms".
- 5) Optional: Call WebXs (Web Access).

mapp hierarchy - Part 3

Exercise: Set up the mapp hierarchy - Part 3

The goal of this exercise is to set up a complete machine. The hierarchies from part 1 (ice cream machine) and part 2 (filling machine) are used for this. The higher-level visualization for the complete machine should display all of the ice cream machine's alarms and bundles the filling machine's alarms.

First the program is created and the mapp AlarmX component is added.

- 1) Under "mapp Services", add the file "MpAlarmXCore" and name it "LineAla".
- 2) Rename MpLink to "MpLinkLINEgAlarmXCore".
- 3) Add an "ST Program All In One" to the project and name it "LineAlarms".
- 4) Use the function block "MpAlarmXCore_0" in the initialization subroutine (enable, MpLink).
- 5) Call the function block in the cyclic program section.
- 6) Use and call the function block "MpAlarmXCore_0" in the exit program.

An HMI application is then created for the complete machine.

- 1) On "ContentALine", add the "AlarmList" widget with AlarmListItems, and connect it to MpLink "MpLinkLINEgAlarmXCore" for the complete machine.
- 2) Optional: Check the binding in the "LineVisu.vis" file.

Edit the AlarmX component of the ice cream machine.

- 1) Alarm mapping \ Mapping 1 - Alarm "TemperatureHigh", escalate alarm
- 2) Alarm mapping \ Mapping 2 - Alarm "TemperatureLow", escalate alarm

Edit the AlarmX component of the filling machine.

- 1) Alarm list \ Alarm - "BottlingProblem", Alarm at the bottling machine!, Persistent
- 2) Alarm mapping \ Mapping 2 - Alarm "BottVolumeChanged", Aggregate alarm and escalate, BottlingProblem

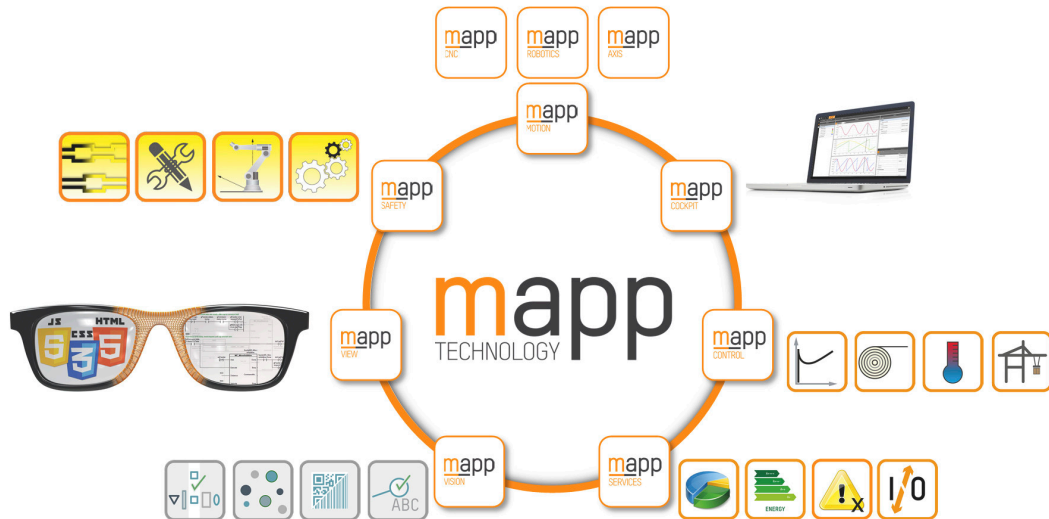
- 3) Alarm mapping \ Mapping 3 - Alarm "ChangeBottling", Aggregate alarm and escalate, BottlingProblem
- 4) Alarm mapping \ Mapping 4 - Alarm "BottlingProblem", Remain

At the end, the complete machine has been configured.

- 1) Under "mapp Services", add the "MpComGroup" file and name it "LineGrp3".
- 2) Rename MpLink to "MpLinkLINEgGroup".
- 3) Under "Child components", the MpLinks of the individual machine groups are entered: MpLinkICEgGroup, MpLinkBOTTgGroup, MpLinkLINEgAlarmXCore.
- 4) The "MpAlarmX" option is selected under "Alarms".

17 Summary

mapp - The future of software engineering



Develop 3x faster

Accelerate your development cycles with modular, intelligent software. mapp components communicate with each other automatically and save you time and money.



Lower lifecycle costs

Benefit from a continuously growing range of functions. New and existing mapps are fully compatible. Leave the software maintenance up to B&R.



Gain time for innovation

Focus on optimizing your machine processes and give innovation the attention it deserves. With mapp Technology, we free up the resources you need to make it happen.



Use the latest technology

mapp Technology makes complex technology like robotics and advanced control technology easy to master.

mapp Technology - Guaranteed software quality

mapp components are implemented according to agile software development methods. The focus is on the quality of the software. Automated tests can be run ahead of time during the development process using test-driven development. Tests are performed at five different levels and new tests are added all the time. Additionally, each new or modified function is developed according to the two-man rule. All of these practices contribute to guaranteeing high-quality software.

Automation Academy

Gain additional knowledge

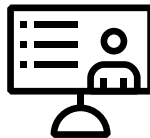
The Automation Academy provides **targeted training** courses for our customers as well as for our own employees. Expand your skills in the field of automation technology and learn to independently implement **efficient automation solutions** using B&R systems.

Decide for yourself which **learning concept** you prefer!



Classroom training

An experienced trainer guides you through the learning program. On-site at the desired B&R location. Learn individually or in small groups.



Virtual classroom

A location-independent distance learning program complements the other learning options we offer. An online tutor accompanies you virtually. The emphasis is on self-study.



Online courses

You acquire your knowledge independently and determine the pace and content yourself. Online courses are available at any time and are independent of duration and location.

Contact

Would you like additional training? Are you interested in finding out what the B&R Automation Academy has to offer? If so, this is the right place.

Access additional information here:

<https://www.br-automation.com/de/academy/>

Enjoy your next training course!



AutomationAcademy





B&R
Industrial Automation GmbH
A member of the ABB Group
B&R Straße 1
5142 Eggelsberg, Austria
office@br-automation.com

t +43 7748 6586-0
f +43 7748 6586-26

br-automation.com

